BigBazaar Architecture Diagrams

System architecture, components, data flow, database schema and request flows

Based on SRS v1.1 and the Phase 1 code on `main` · 2026-10-13

Contents

1 **System architecture**
- 1.1 Target system architecture
- 1.2 Deployment and load balancing
- 1.3 Network diagram (production)
- 1.4 Recommended cloud setup and firewall rules

2 **Component diagrams**
- 2.1 Identity service internals (the template for every Java service)
- 2.2 Angular web app

3 **Data-flow diagrams**
- 3.1 Level 0: context diagram
- 3.2 Level 1: main processes and data stores
- 3.3 Event flow: who publishes and who listens

4 **Database schema (ERD)**
- 4.1 Overview: databases and the ids they borrow
- 4.2 identity_db (Identity)
- 4.3 catalog_db (Catalog)
- 4.4 commerce_db (Commerce)
- 4.5 payment_db (Payment and settlement)
- 4.6 fulfillment_db (Fulfillment)
- 4.7 notification_db (Notification)
- 4.8 trust_db (Trust)
- 4.9 recommendation_db (Recommendation)
- 4.10 insights_db (Insights)
- 4.11 platform_db (Audit, support tickets)
- 4.12 Search index (OpenSearch)
- 4.13 Every table and every column

5 **Request-flow diagrams**
- 5.1 Any request through the gateway
- 5.2 Login with MFA
- 5.3 Checkout and payment saga
- 5.4 Seller verification
- 5.5 Product publication
- 5.6 Shipping and delivery
- 5.7 Return and refund
- 5.8 Settlement and payout
- 5.9 Review and trust
- 5.10 Outbox and idempotent consumer

6 **Choices made and alternatives**

7 **Open items found while drawing**

8 **Glossary**

9 **Appendix: Mermaid source**

How to read this document

This document draws BigBazaar from five angles. Each angle answers a different question, and together they describe the whole system before we build Phase 2.

- **System architecture**: what are the big boxes, and how do they talk?
- **Components**: what is inside one box?
- **Data flow**: where does information come from and where does it end up?
- **Database schema (ERD)**: what tables does each service keep, and how are they linked?
- **Request flows**: step by step, what happens when someone logs in, buys, ships or returns?

Every diagram is written in **Mermaid**, a text format for diagrams. The text lives in `docs/diagrams/mermaid/`, so a diagram changes in a pull request like code does, and GitHub draws it in the browser. The full source of every diagram is printed in the appendix.

What is built and what is planned. Phase 1 built the gateway, Identity and Audit. Everything else is designed here from SRS v1.1 and will be built in later phases. In the schema tables a green **Built** badge means the table exists in a Flyway migration today; **Planned** means it is designed from SRS 8.4; an amber badge marks the few tables these diagrams add that SRS 8.4 does not name, each with a note saying why it is needed.

Five ideas behind every diagram

These ideas explain most of the lines you will see.

- **Microservice with its own database.** Each service (Identity, Catalog, Commerce and so on) owns its tables. No other service reads them or points a foreign key at them (SRS 3.5). That keeps each service free to change its own tables without breaking others.
- **Logical id instead of a foreign key.** When Commerce stores a `seller_id`, it is just a UUID copied from Identity, not a database link. The ERDs show these as "logical id"; the overview page shows which database borrows ids from which.
- **Projection.** A service that needs another service's data keeps its own read-only copy, built from events. Catalog's `verified_seller_projection` is the list of approved sellers, rebuilt from `SellerApproved` and `SellerSuspended` events. This avoids calling Identity on every product save.
- **Transactional outbox.** A service saves its change and an "event to send" row in the same database transaction, and a relay sends the row to Kafka afterwards. So an event is published if and only if the change was saved (ADR 0002).
- **Idempotent consumer.** Kafka may deliver an event twice. Each consumer remembers the event ids it has handled in `processed_events` and skips repeats, so doing the work twice can never double a refund or a stock change.

1. System architecture

The big picture: which pieces exist and how they connect. Two kinds of arrows matter. **Solid REST arrows** are a question that needs an answer now ("log me in", "show this product"). **Kafka arrows** are announcements of facts that already happened ("order created"), which any interested service can react to later.

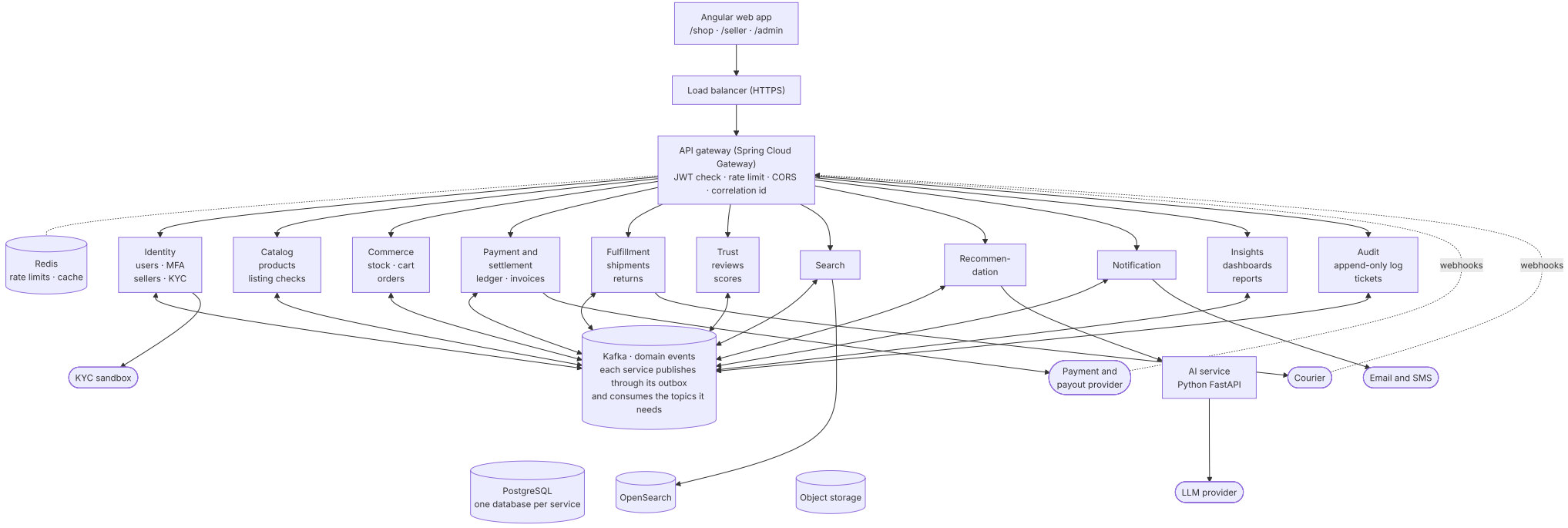
1.1 Target system architecture

From SRS v1.1: 3.1 Target architecture, 3.4 Communication rules, 7 Microservice requirements, 14 External interfaces

Read top to bottom. The one Angular app sends every request through the load balancer to the API gateway, which checks the token, applies rate limits and routes to one of eleven Java services. Each service writes to its own PostgreSQL database and announces facts on Kafka through its outbox. The bottom row is everything outside our control, always reached through an adapter so a mock can stand in during development.

Key ideas

- **Why a gateway?** One front door means the browser knows one address, and token checks, rate limits, CORS and correlation ids are done once instead of in every service.
- **Why the AI service is separate and in Python.** Only Recommendation calls it, and Python has the best AI tooling. Keeping it behind Recommendation means the AI can never invent a product: it only ranks and explains real catalog candidates (FR-AI-02).
- **Webhooks** are calls the payment provider and courier make to us. They enter through the gateway like any other request and are verified and deduplicated by the owning service.



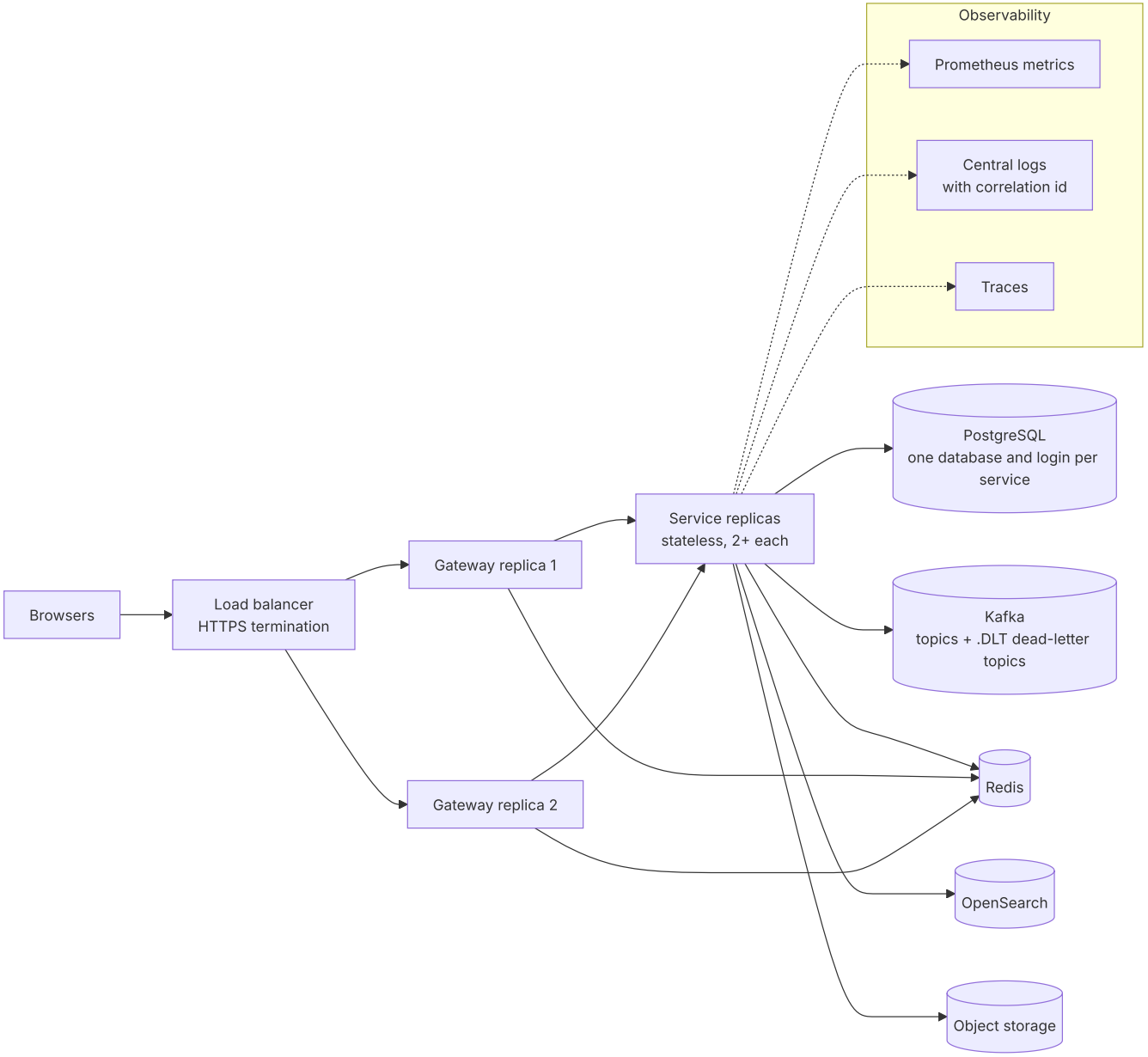
Source: `mermaid/01-system-architecture.mmd`

1.2 Deployment and load balancing

From SRS v1.1: 15 Deployment and operations, 15.2 Load balancing, 6.3 Availability, 6.6 Observability

How it runs in production. Every service is stateless, so two or more copies run behind the load balancer and any copy can serve any request. State lives only in the data stores on the right.

> **Key ideas**
>
> - **Stateless** means a service keeps nothing in memory between requests. That is what lets us add copies under load and lose one without losing data.
> - **.DLT topics** (dead-letter topics) hold events a consumer failed to process after retries, so one bad event cannot block the rest and can be inspected later.

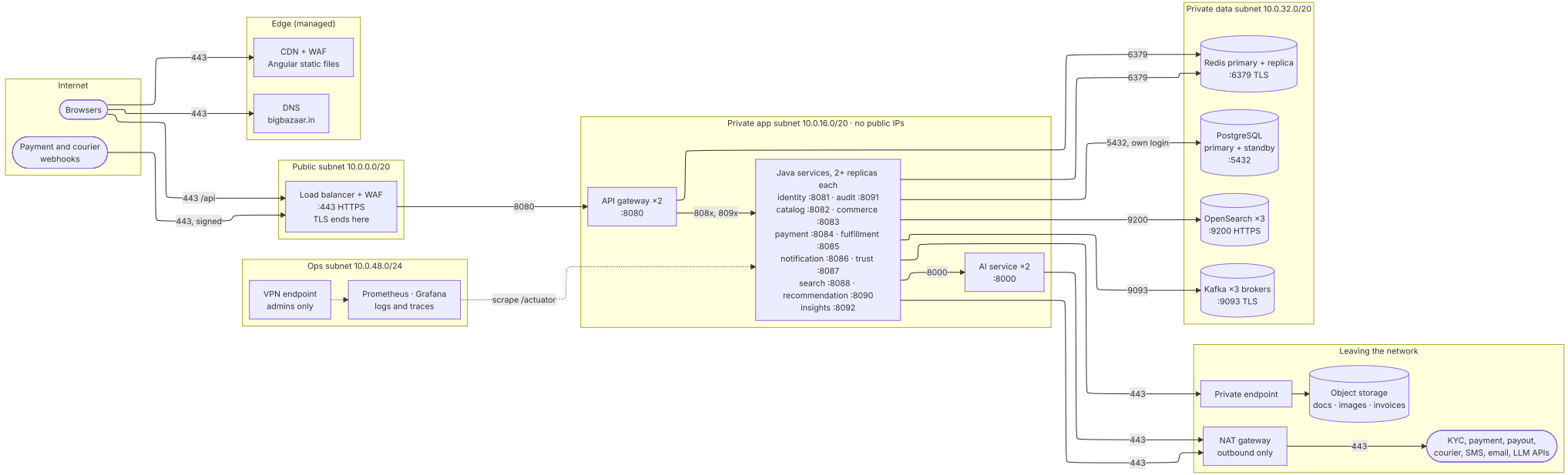


Source: `mermaid/02-deployment.mmd`

1.3 Network diagram (production)

From SRS v1.1: 12 Security requirements, 15 Deployment, 15.2 Load balancing, 6.3 Availability

Read left to right, from the internet inward. Each box is a network zone and each arrow is a connection that a firewall rule allows, labelled with its port. Anything without an arrow is blocked. Only the load balancer has a public address. Services sit in a private subnet, databases in a deeper one with no internet route at all, and calls out to providers leave through one NAT gateway. Every zone is copied across two availability zones (separate data centres). Ports for services not built yet are proposed; the built ones (gateway 8080, identity 8081, audit 8091) match the code.



Source: `mermaid/08-network.mmd`

1.4 Recommended cloud setup and firewall rules

From SRS v1.1: 12 Security requirements, 14 External interfaces, 15 Deployment

Recommendation: AWS, Mumbai region (ap-south-1). The storage adapter already defaults to S3 in ap-south-1, Indian payment and KYC rules favour keeping customer data in India, and every box in the network diagram has a managed AWS service, so the team runs code rather than servers. The diagram keeps generic names so it stays true if another cloud is chosen (Azure Central India and GCP asia-south1 have the same pieces, but no managed Kafka on Azure).

Three ideas behind the network diagram: - **Subnet** is a slice of the private network (written as a CIDR range such as 10.0.16.0/20). Putting each tier in its own subnet lets firewall rules say "only the app tier may reach the database". - **Defence in depth**: an attacker who gets past the load balancer still cannot reach a database directly, and a stolen database password is useless from outside the network. - **NAT gateway** lets private machines start connections to the internet (KYC, payments) while nothing on the internet can start a connection to them.

Diagram box	AWS service	Why
Private network, subnets, firewall rules	VPC over 2 availability zones, security groups	Rules reference groups, not IPs, so scaling never breaks them.
DNS	Route 53	Health-checked records.
CDN + WAF	CloudFront + AWS WAF	Serves the Angular build from S3 near the user.
Load balancer	Application Load Balancer + AWS WAF, ACM certificate	HTTPS ends here; spreads load over gateway replicas.
Gateway, services, AI service	ECS on Fargate, images in ECR	Containers without managing servers; two or more tasks per service.
PostgreSQL	RDS PostgreSQL Multi-AZ	Automatic standby and backups; one login per service database.
Kafka	Amazon MSK (3 brokers, TLS)	Managed Kafka with the same client code.
Redis	ElastiCache for Redis, TLS	Rate limits and cache.
OpenSearch	Amazon OpenSearch Service (3 nodes)	Same engine the code targets.
Object storage + private endpoint	S3 + VPC gateway endpoint	File traffic never leaves AWS's network.
NAT gateway	NAT gateway in each public subnet	Outbound calls to providers only.
Monitoring	Managed Prometheus + Grafana, CloudWatch logs	Metrics, logs and traces with the correlation id.
VPN endpoint	AWS Client VPN (or SSM Session Manager)	Admins reach ops tools without any public SSH.

From	To	Port	Allowed because
Internet	Load balancer	443	The only public entry: browsers and signed webhooks.
Load balancer	API gateway	8080	Gateway checks tokens and routes.
API gateway	Services	8081 to 8092	REST calls the gateway routes.

From	To	Port	Allowed because
API gateway, services	Redis	6379	Rate limits and cache.
Recommendation	AI service	8000	Only Recommendation calls the AI.
Each service	PostgreSQL	5432	Each with its own login to its own database.
Services	Kafka	9093	Outbox relays publish, listeners consume.
Search	OpenSearch	9200	Indexing and queries.
Services	Object storage endpoint	443	Documents, images, invoices, reports.
Services, AI service	NAT gateway → providers	443	KYC, payment, courier, SMS, email, LLM.
Monitoring	Services	actuator port	Metrics scraping.
Anything else	Anything	none	Denied by default.

2. Component diagrams

A component diagram opens one box from the architecture and shows the parts inside it. We draw the Identity service, because it is built and every later Java service will follow the same layers, and the Angular web app, which kai chose to build as one app with three areas.

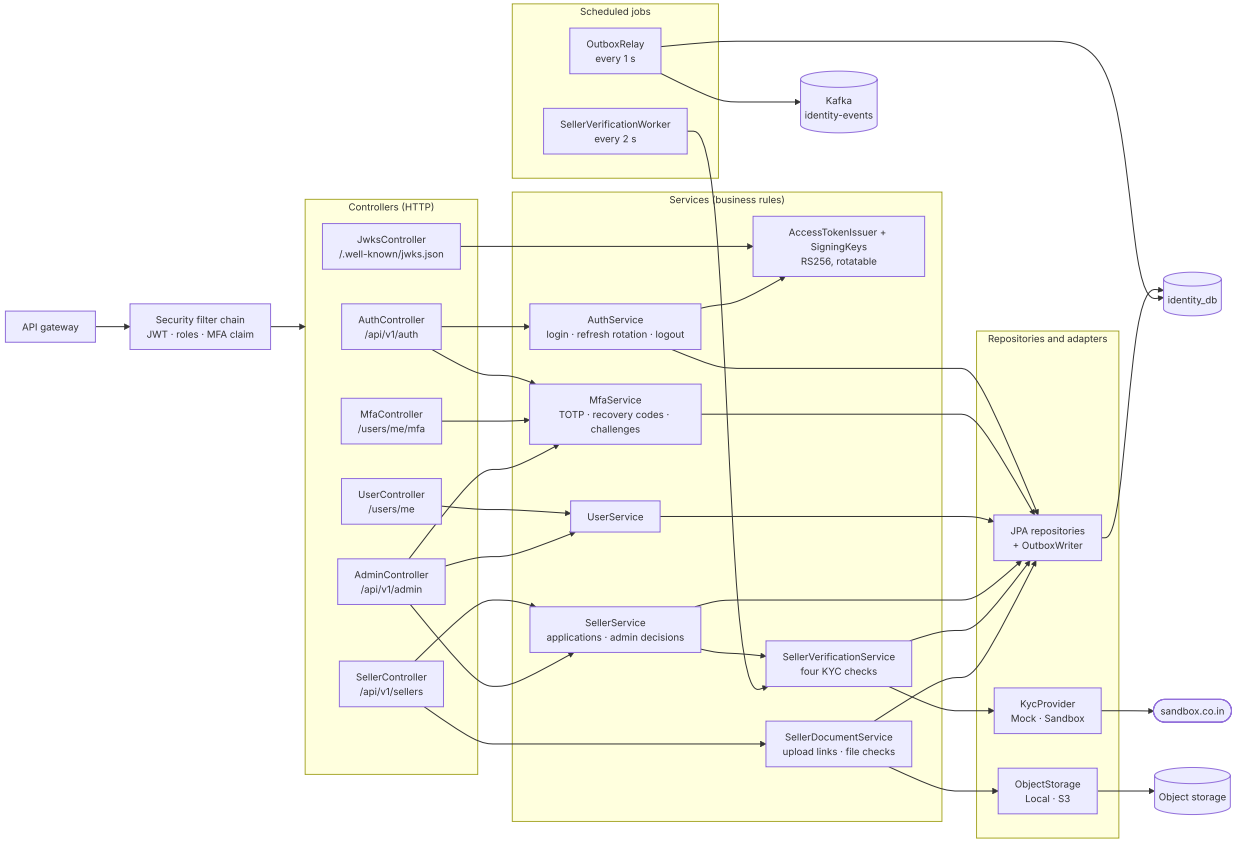
2.1 Identity service internals (the template for every Java service)

From SRS v1.1: 7 Microservice requirements, 9.2 Outbox, 12 Security; code under services/identity-service

Requests come in at the top through the security filter, reach a controller, which calls a service class holding the business rules, which saves through repositories. Anything that talks to the outside world (KYC provider, file storage, Kafka) sits behind an adapter at the bottom. Two scheduled jobs run on their own: the verification worker and the outbox relay.

Key ideas

- **Layers.** Controller (HTTP in and out) → service (rules) → repository (database). Rules never sit in controllers, so they can be tested without HTTP.
- **Adapter.** An interface such as `KycProvider` with a mock and a real implementation, picked by configuration. Catalog, Payment and Fulfillment will reuse this for images, payments and couriers.
- **`libs/common`** holds what every service must agree on: the event envelope, topic names, the error response and header names.



Source: `mermaid/03-component-identity.mmd`

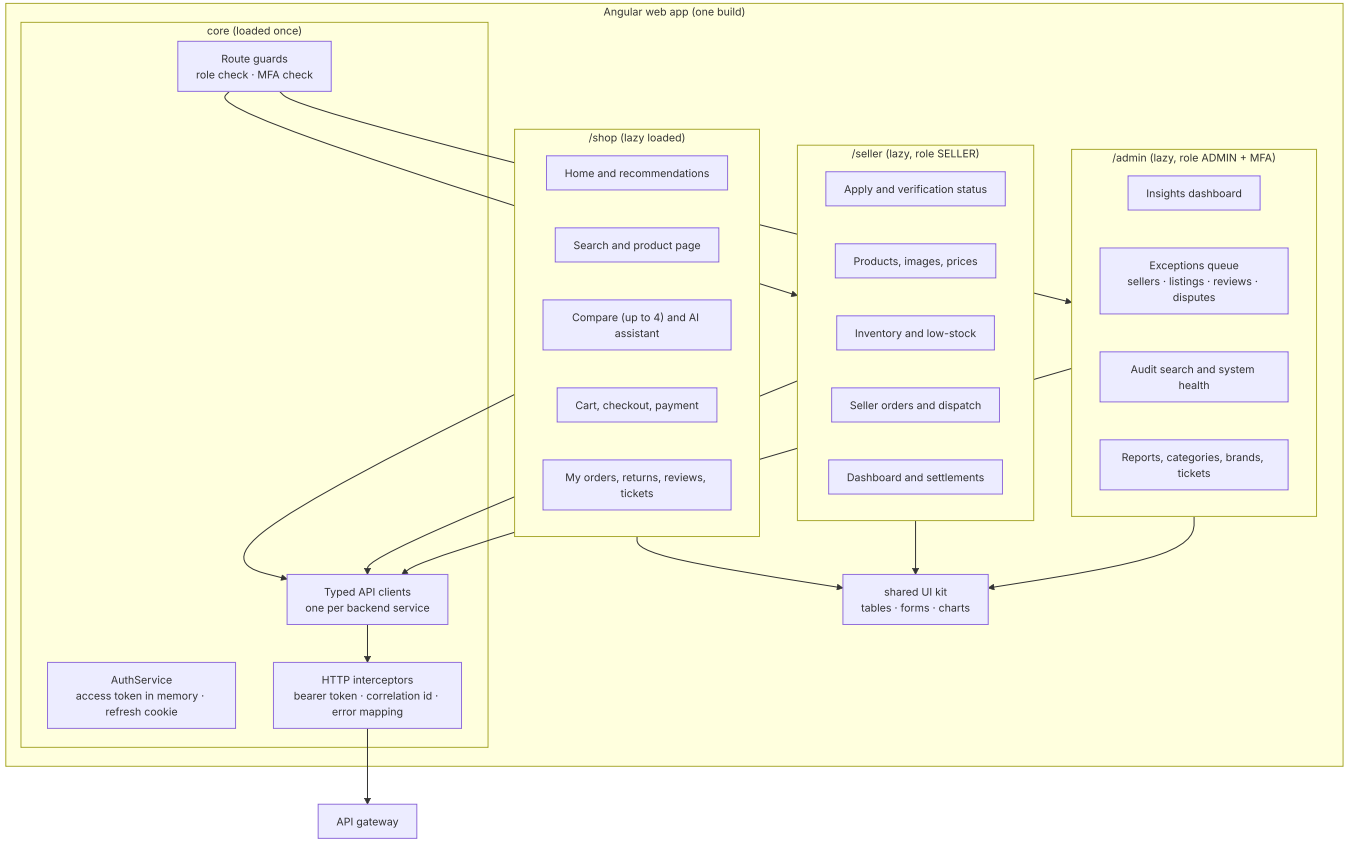
2.2 Angular web app

From SRS v1.1: 13 User interface requirements (13.1 Shop, 13.2 Seller center, 13.3 Admin insights)

One Angular build with three lazily loaded areas: `/shop` for everyone, `/seller` for sellers and `/admin` for admins. The core layer is loaded once and shared: it holds the login state, attaches the token to every call and blocks routes the user may not open.

Key ideas

- **Lazy loading** means a shopper's browser never downloads the seller or admin code.
- **Route guards are for convenience, not security.** The real check is always on the server; the guard only stops the app from showing a page the server would refuse.
- **Admin has no order or listing controls.** Admin sees insights and decides only exceptions (SRS 4.1), so the admin area is dashboards plus an exceptions queue.



Source: `mermaid/04-component-web-app.mmd`

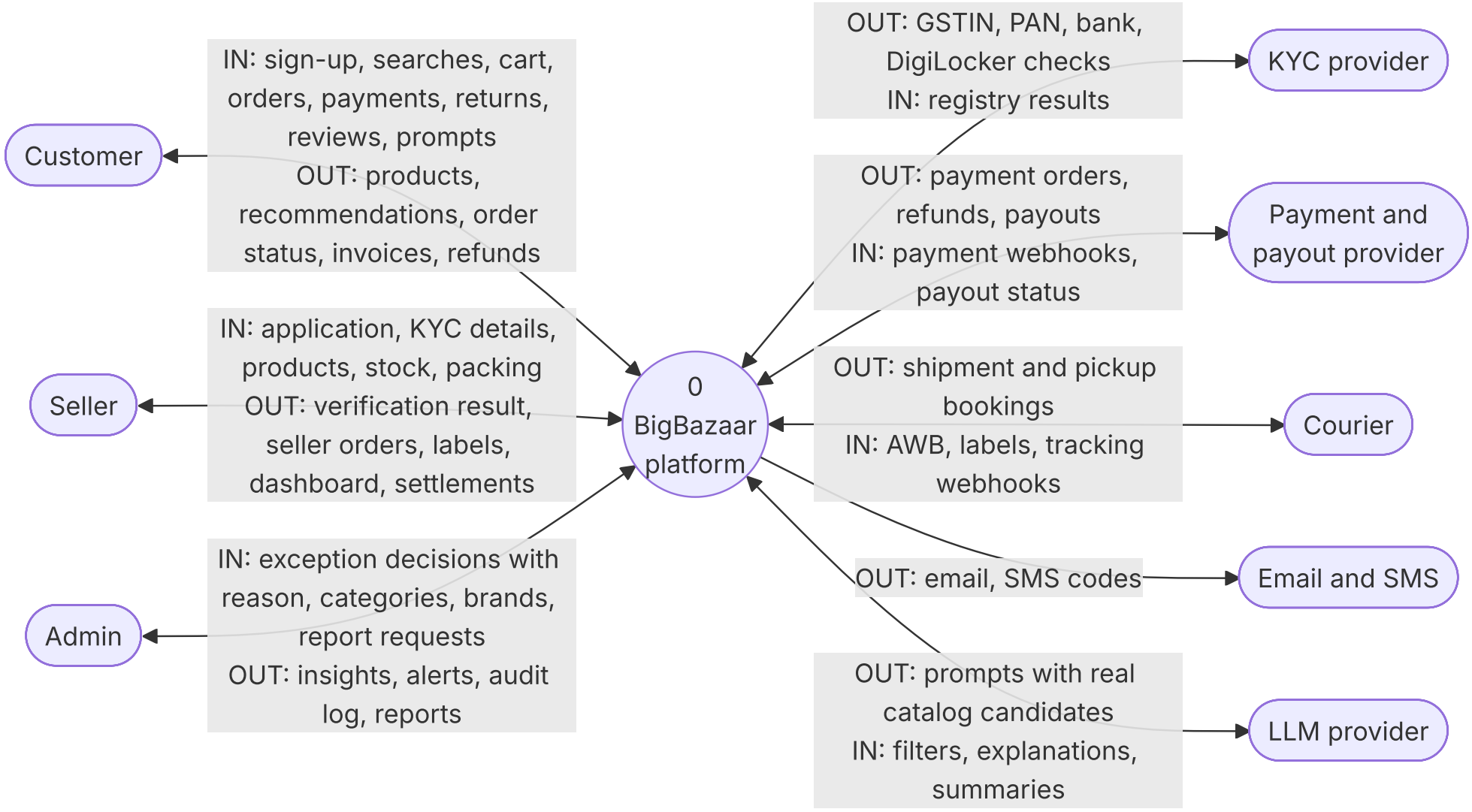
3. Data-flow diagrams

A data-flow diagram (DFD) ignores servers and code and follows the information. Rounded boxes are people or outside systems, rectangles are processes that change data, cylinders are data stores and arrows are labelled with the data that moves. Level 0 shows the whole system as one process; level 1 opens it into its main processes.

3.1 Level 0: context diagram

From SRS v1.1: 2.1 Product perspective, 4 User classes and actors, 14 External interfaces

Everything BigBazaar receives and sends, and from whom. Three human roles (customer, seller, admin; there is no support role in v1.1) and five outside providers.

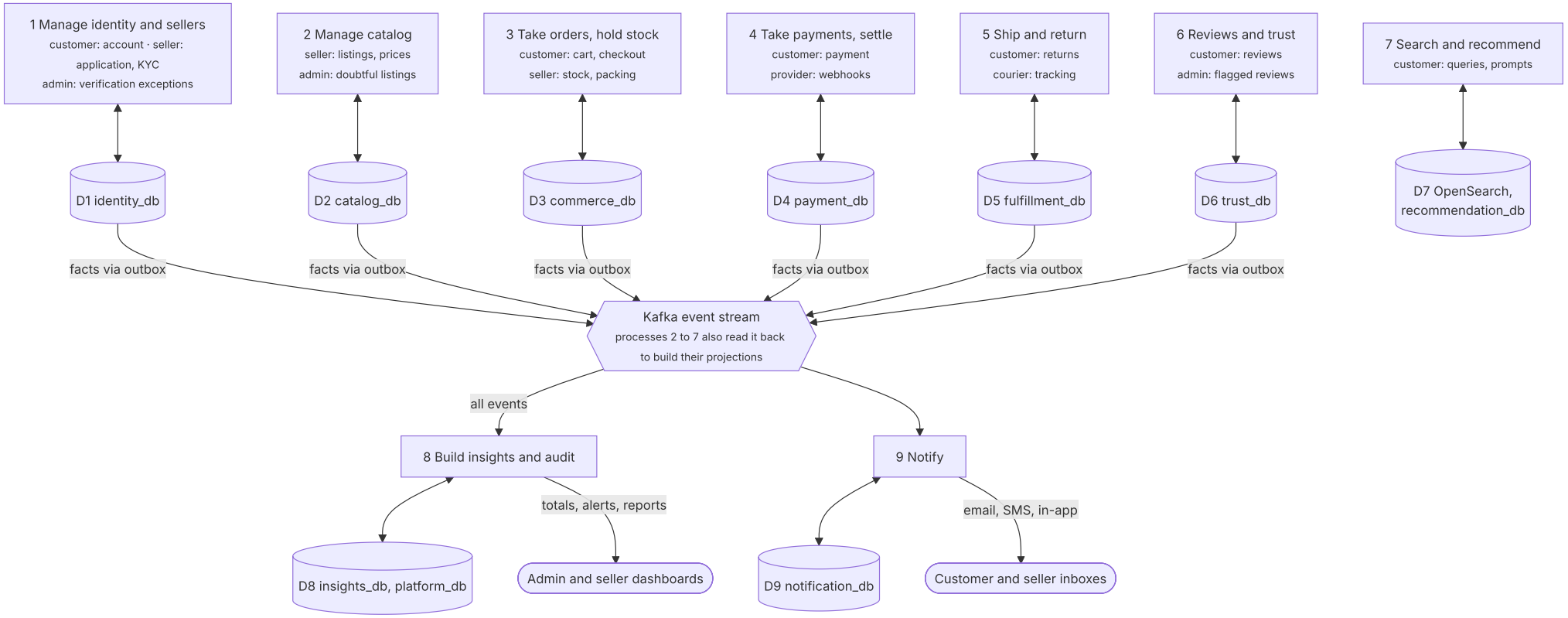


Source: `mermaid/05-dfd-context.mmd`

3.2 Level 1: main processes and data stores

From SRS v1.1: 3.5 Data ownership, 5 Functional requirements, 9 Event-driven architecture

Each numbered process is roughly one service and owns exactly one data store (D1 to D9). The small text in each process says who feeds it, from the context diagram. Processes never reach into each other's stores. Instead, facts leave each store through its outbox as events, and other processes read them back to build their own copies. Insights and Audit read every event, which is why dashboards never query the other databases.



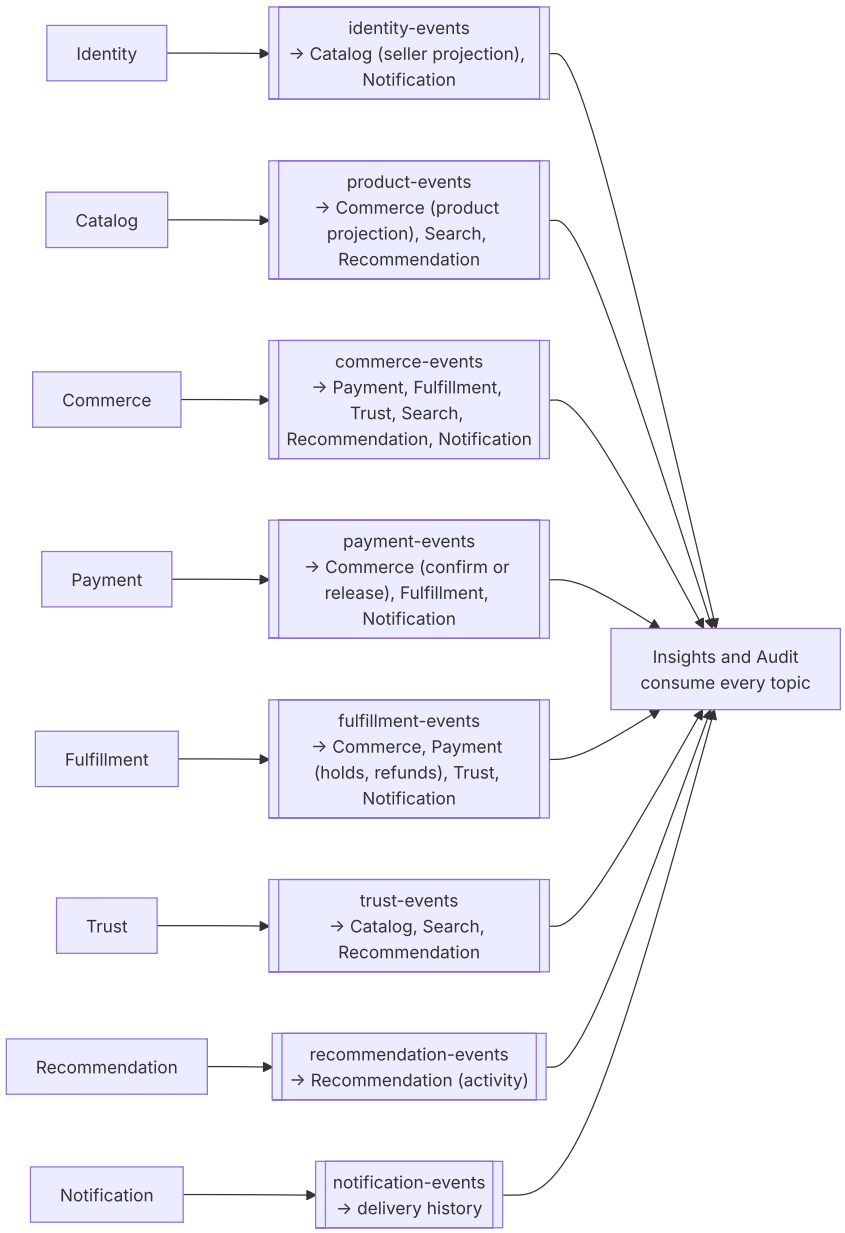
Source: `mermaid/06-dfd-level1.mmd`

3.3 Event flow: who publishes and who listens

From SRS v1.1: 9.3 Event catalog, 9.4 Consumer requirements

Each service publishes to exactly one topic named after it. Read a consumer box to see what it builds from the events it receives. Insights and Audit subscribe to every topic.

> **Key ideas**
>
> - **One topic per producer** keeps ownership obvious: only Commerce writes `commerce-events`.
> - **Message key = entity id** (for example the order id), so all events about one order arrive in order on the same partition.



Source: `mermaid/07-event-flow.mmd`

4. Database schema (ERD)

An entity-relationship diagram (ERD) shows tables as boxes and foreign keys as lines. As agreed, the diagrams show only the **main columns** (keys, status and the fields that explain the table) so each page stays readable. **Every column**, with type, nullability, keys and notes, is in the column tables at the end of this part.

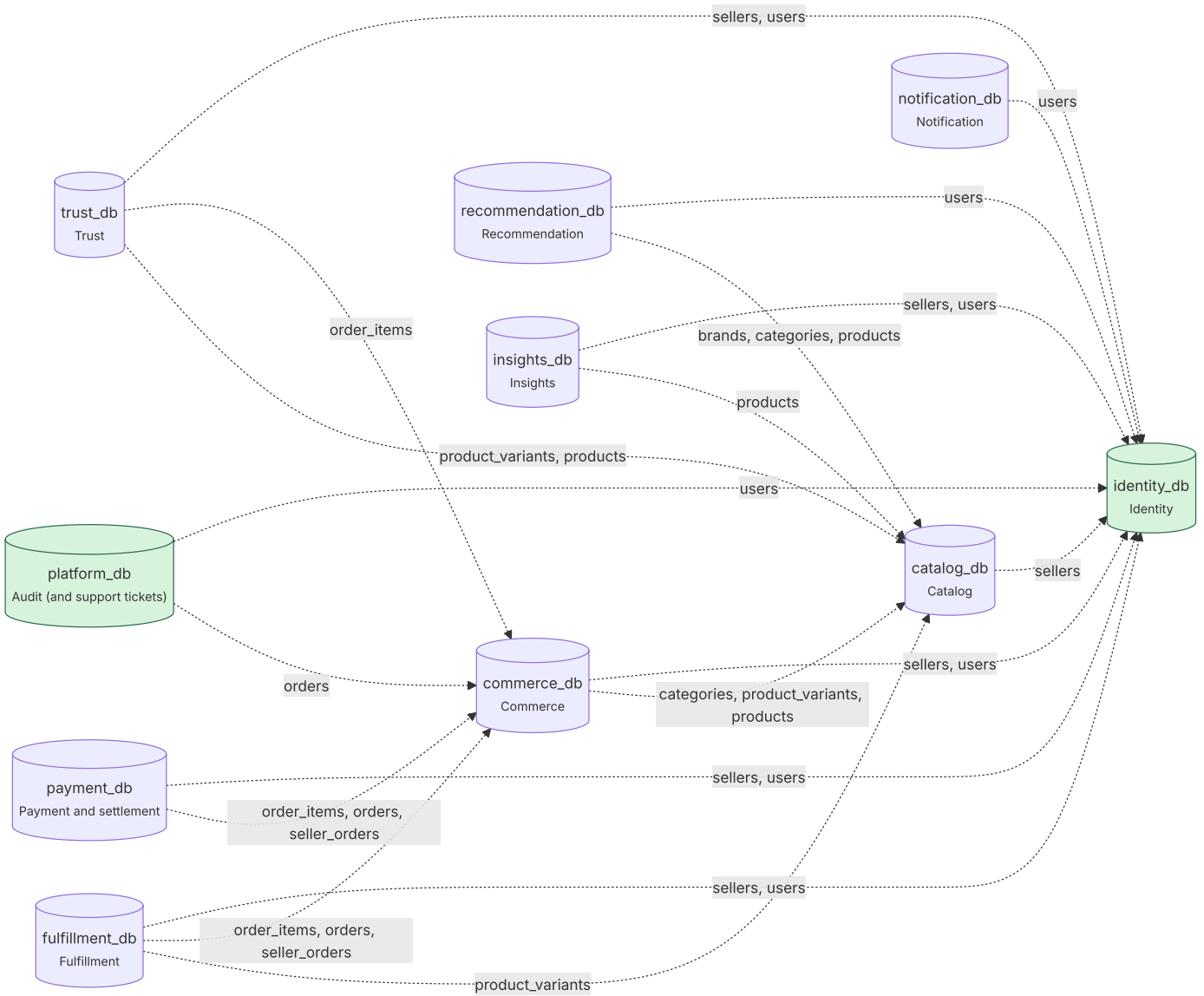
How to read the lines (crow's foot notation): `||` means exactly one, `|o` zero or one, `o{` zero or many. So `users ||--o{ addresses` reads "one user has zero or many addresses". Lines exist only inside one database. A column marked "logical id" holds an id owned by another service, with no foreign key.

Conventions from SRS 8.2 apply everywhere: UUID ids (BIGINT for high-volume append-only logs), money as `numeric(12,2)`, times as `timestamptz`, enums stored as strings, and `deleted_at` soft delete on the tables that allow it. History tables (status history, stock movements, ledger entries, audit events) are append-only: a database trigger refuses updates and deletes.

4.1 Overview: databases and the ids they borrow

From SRS v1.1: 3.5 Data ownership, 8.1 Database strategy

One cylinder per database (green ones exist today). A dotted arrow from A to B means A stores ids of B's records, as plain values. For example Commerce stores `users` and `sellers` ids from Identity.

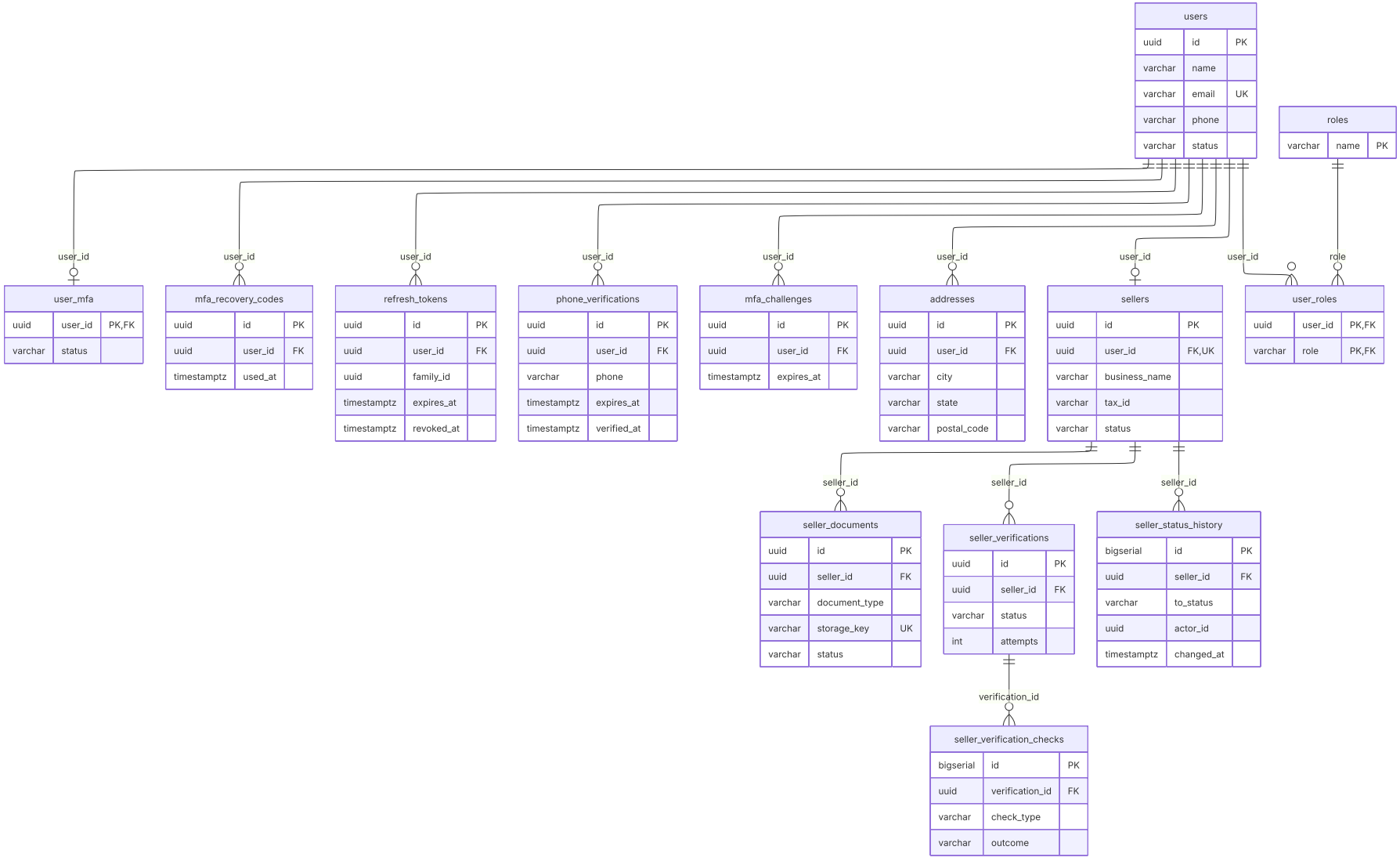


Source: `mermaid/20-erd-00-overview.mmd`

4.2 identity_db (Identity)

From SRS v1.1: 8.4; migrations V1 to V4

Built in Phase 1. Users hold roles through `user_roles`. MFA adds `user_mfa`, recovery codes and login challenges. A user can apply to become a seller; each application attempt is a `seller_verifications` row with one `seller_verification_checks` row per KYC check. `phone_verifications` is the only planned table here (FR-AUTH-09).

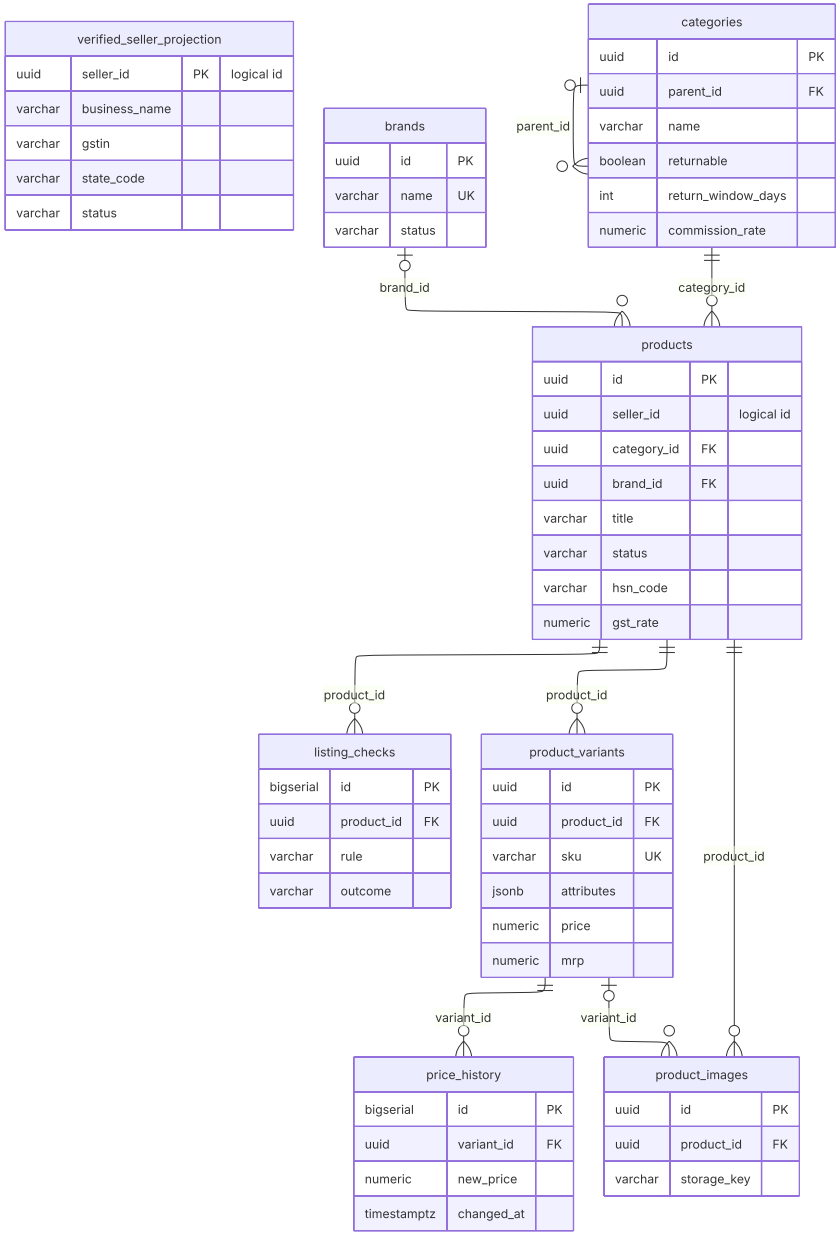


Source: `mermaid/20-erd-01-identity-db.mmd`

4.3 catalog_db (Catalog)

From SRS v1.1: 8.4, 5.3 Catalog

A product belongs to a category and optionally a brand, and has one or more variants (the SKUs people actually buy). Categories carry the return window and commission rate. Price changes append to `price_history`; listing checks record why a product went live, was rejected or went to an admin.

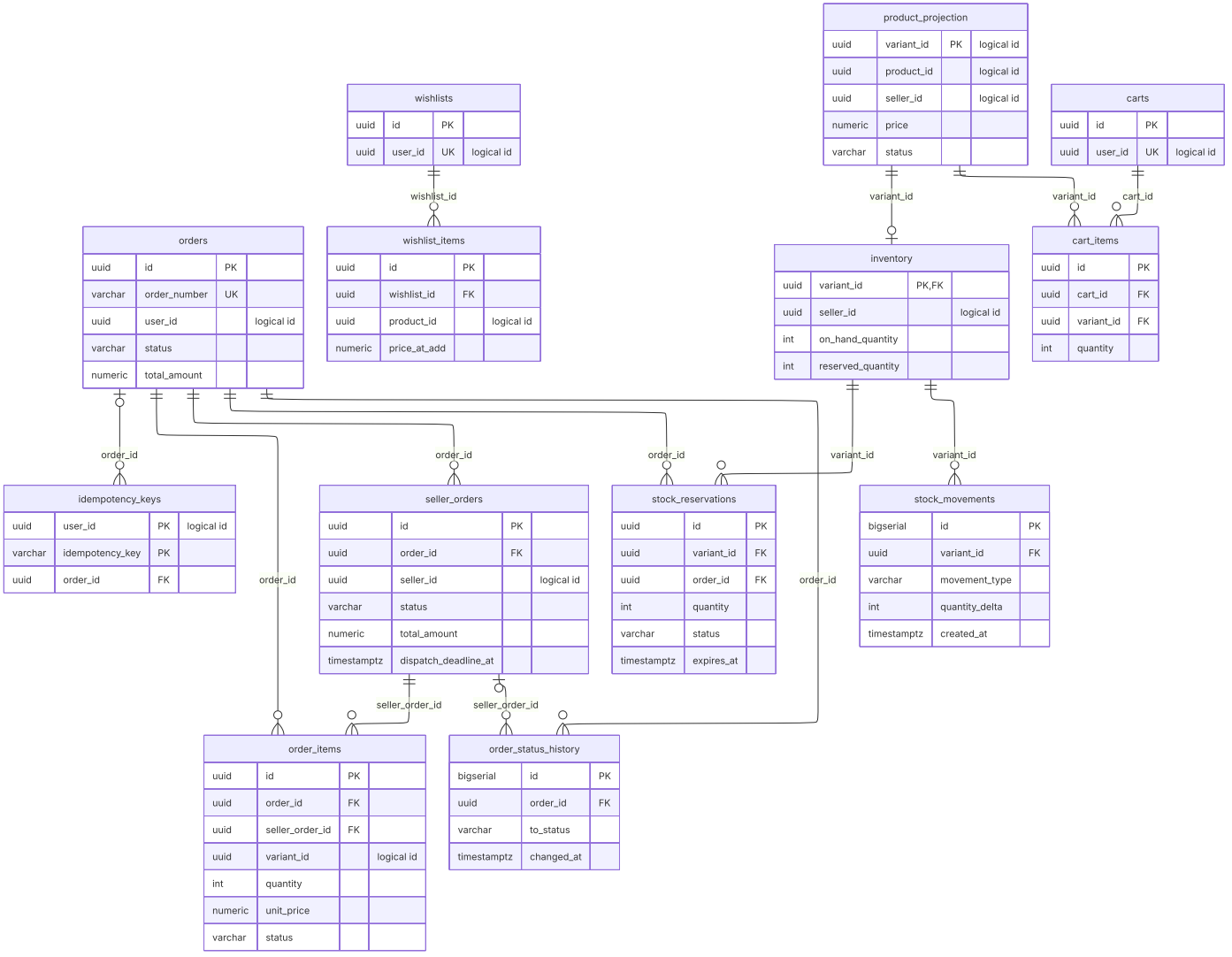


Source: `mermaid/20-erd-02-catalog-db.mmd`

4.4 commerce_db (Commerce)

From SRS v1.1: 8.4, 8.6 Inventory, 8.7 Orders

Inventory and orders live together so that reserving stock and creating the order happen in one transaction. An order splits into one `seller_orders` row per seller; `order_items` copy every price, tax and product detail at purchase time so later edits never change an old order.

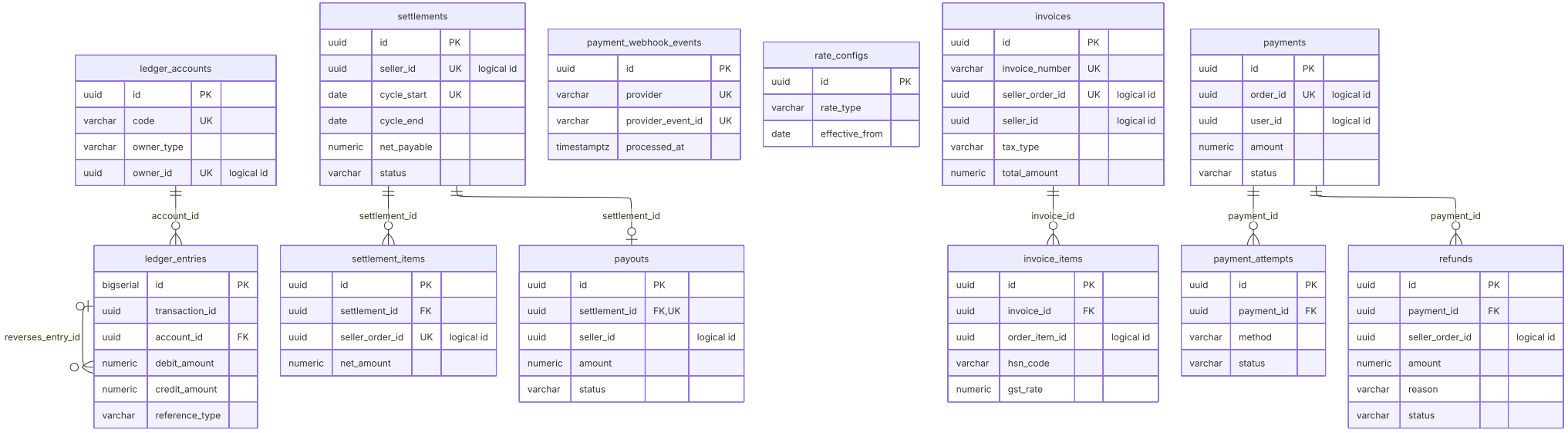


Source: `mermaid/20-erd-03-commerce-db.mmd`

4.5 payment_db (Payment and settlement)

From SRS v1.1: 8.4, 8.8 Ledger, 5.8 to 5.9

Two halves. Payments: `payments`, their attempts, raw webhooks and refunds. Money bookkeeping: the double-entry ledger, settlements per seller per cycle, payouts and GST invoices. The ledger lives here (not in its own service) because a payment and its ledger entries must be saved together.

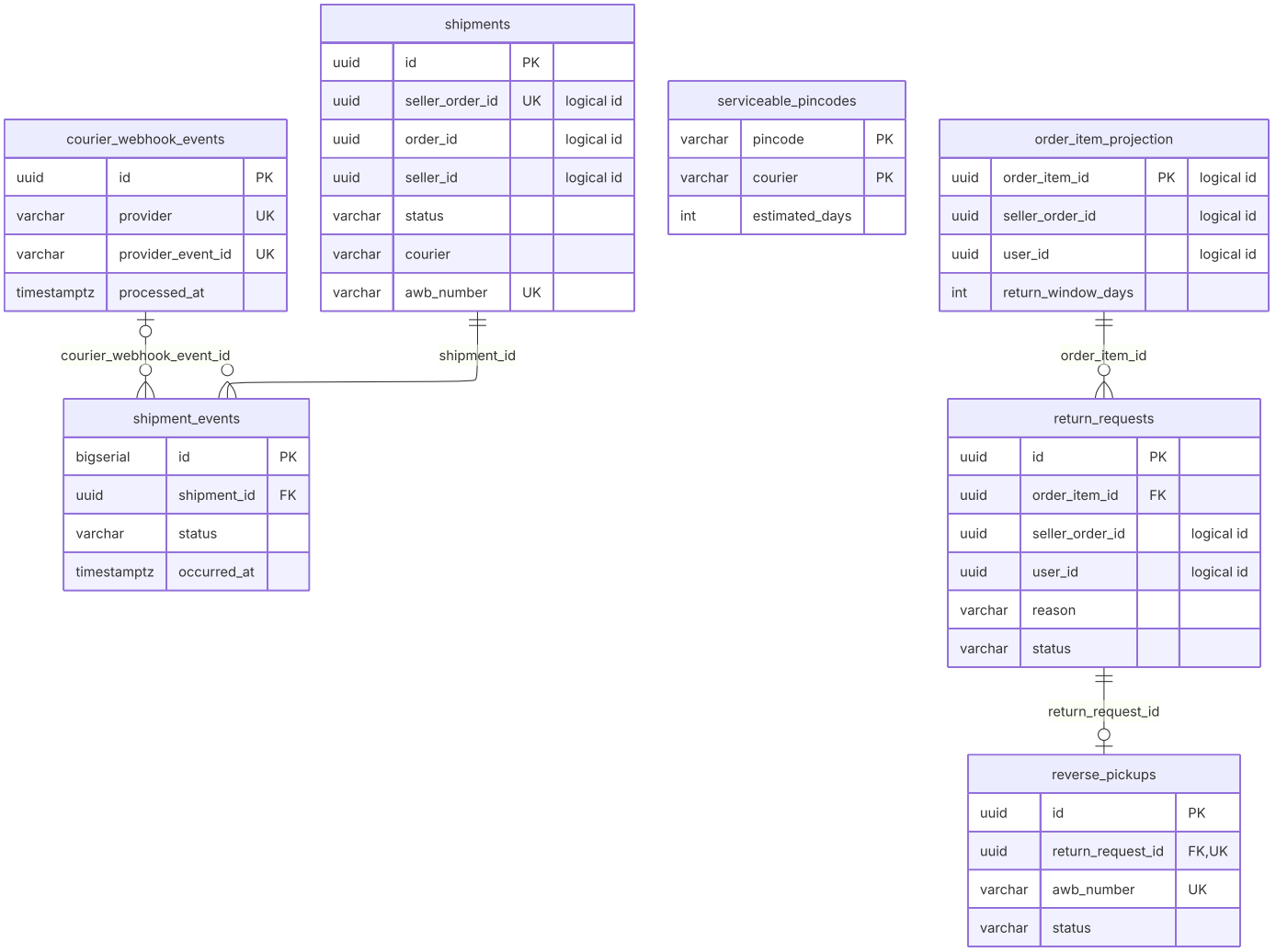


Source: `mermaid/20-erd-04-payment-db.mmd`

4.6 fulfillment_db (Fulfillment)

From SRS v1.1: 8.4, 5.6 Shipping, 5.7 Returns

One shipment per seller order with its tracking timeline, built from deduplicated courier webhooks. Returns are requested per order item and each gets one reverse pickup.

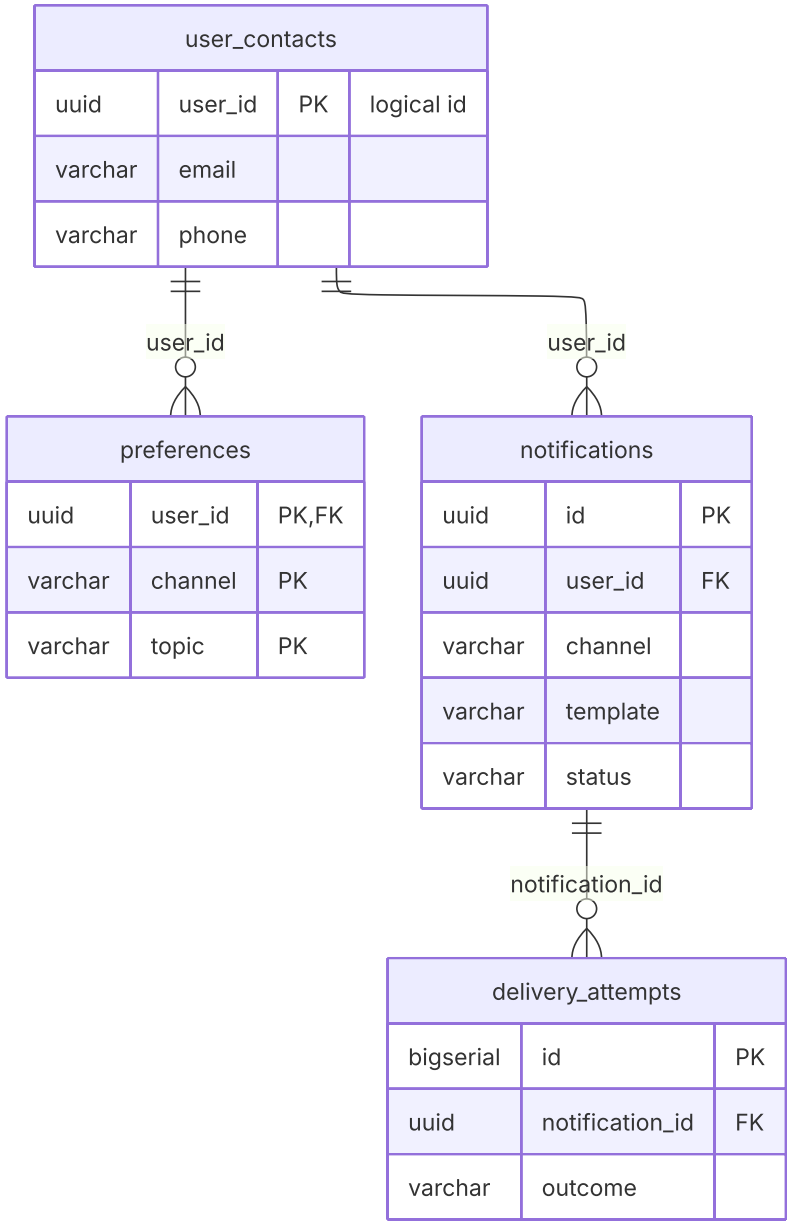


Source: `mermaid/20-erd-05-fulfillment-db.mmd`

4.7 notification_db (Notification)

From SRS v1.1: 8.4, 5.12

Each notification is one message on one channel, with every send attempt logged for retries.

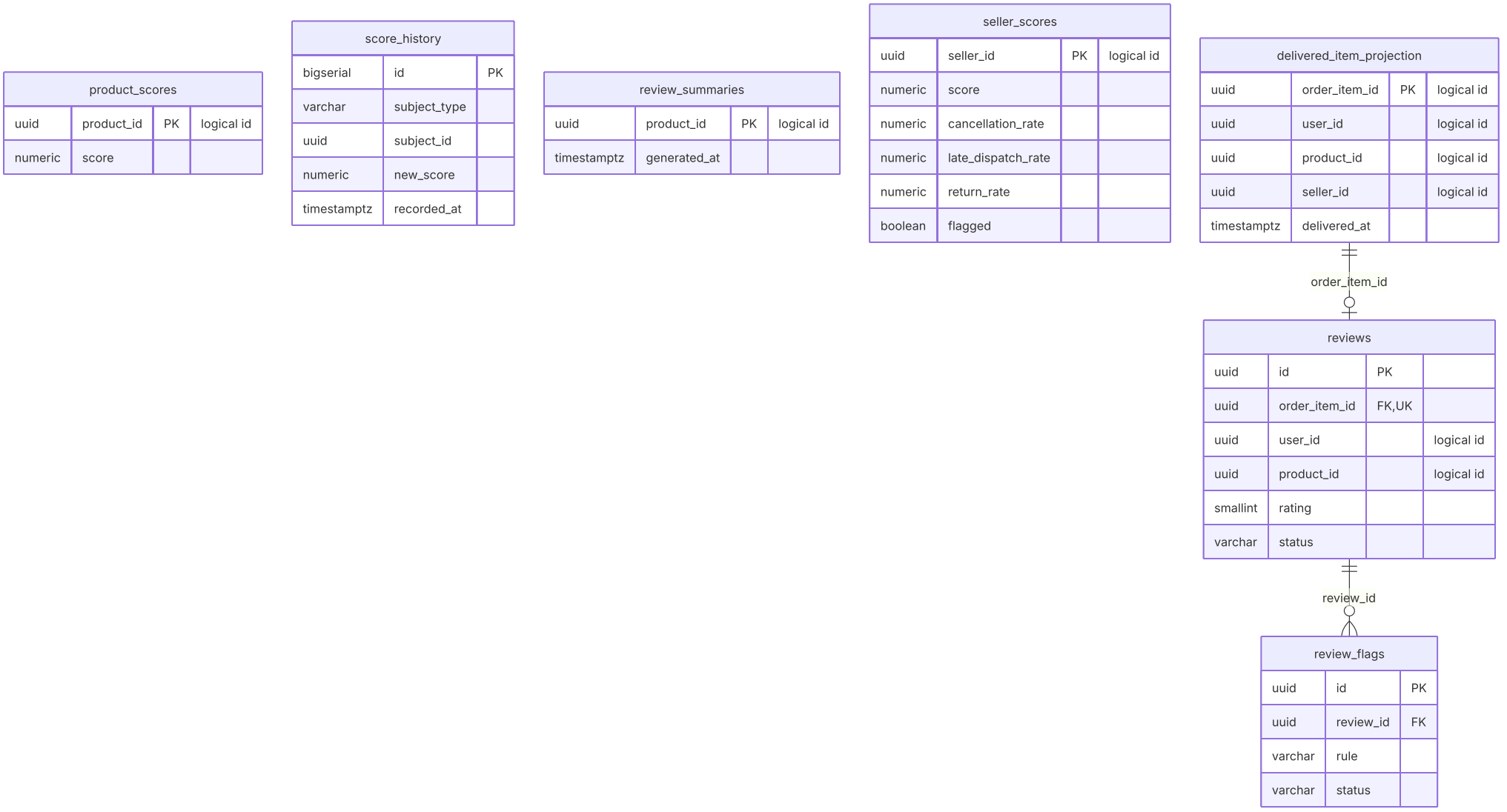


Source: `mermaid/20-erd-06-notification-db.mmd`

4.8 trust_db (Trust)

From SRS v1.1: 8.4, 5.10 Reviews and trust

A review can only exist for a delivered order item, and only one per item, which the one-to-one line from `delivered_item_projection` to `reviews` shows. Scores keep their reasons in `components` and every change in `score_history`.

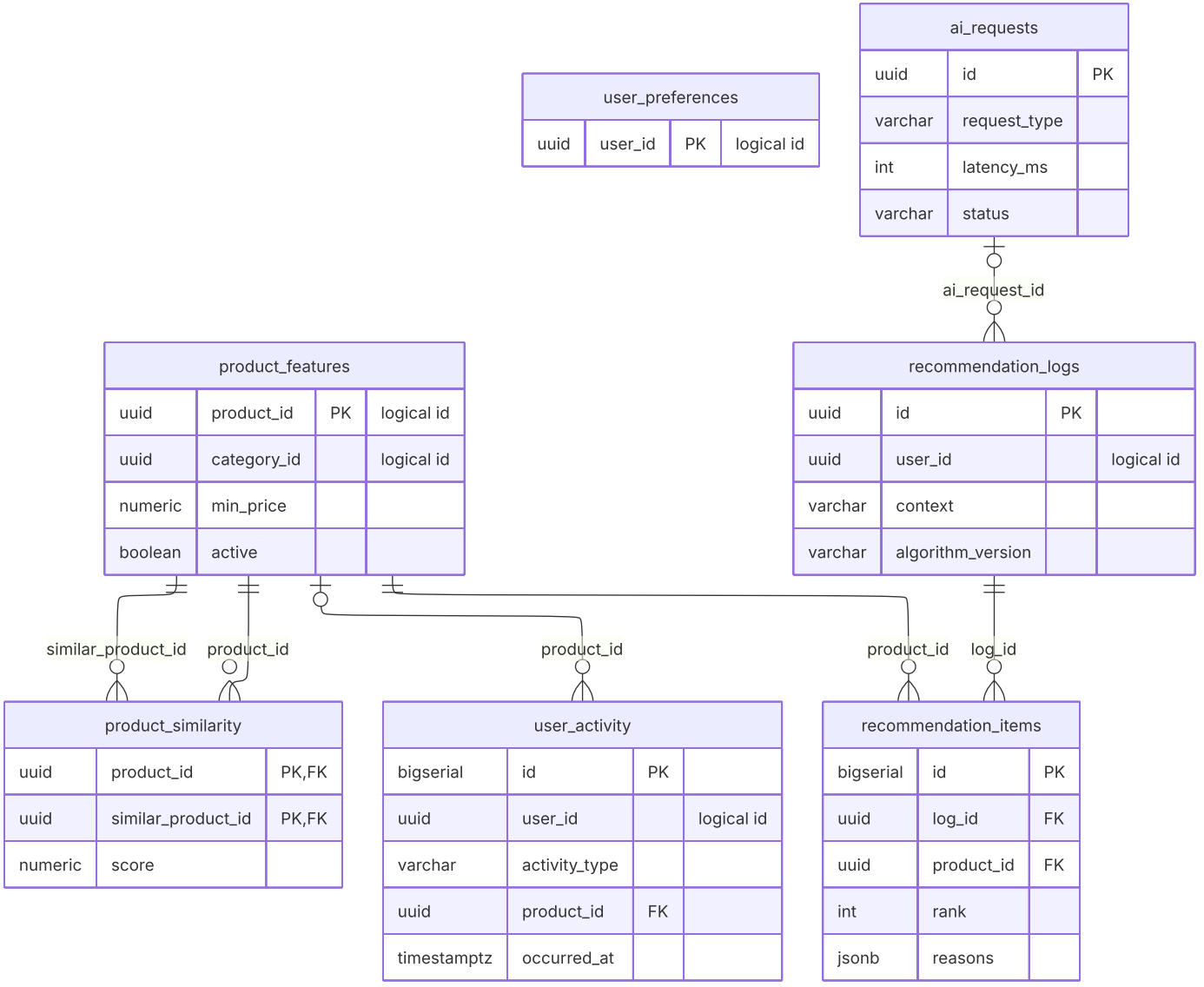


Source: `mermaid/20-erd-07-trust-db.mmd`

4.9 recommendation_db (Recommendation)

From SRS v1.1: 8.4, 5.11, 11 AI and recommendation

Activity goes in, ranked lists come out. Every list is logged with the algorithm version and each item's reasons, and every AI call is logged with its tokens, cost and time (FR-AI-04).

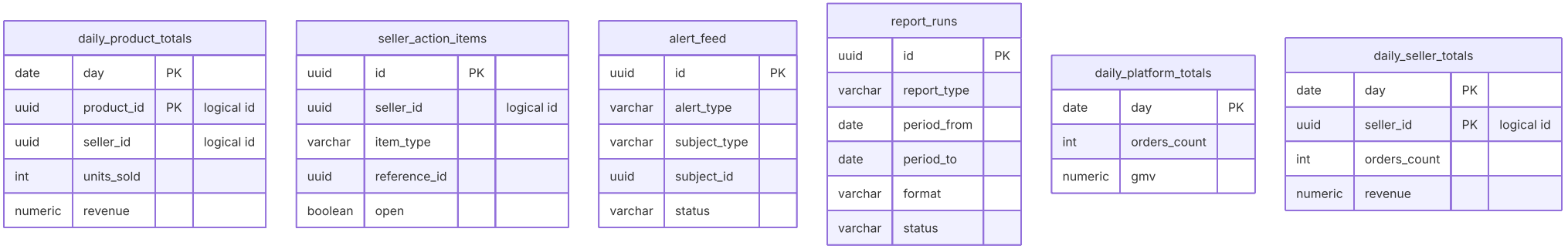


Source: `mermaid/20-erd-08-recommendation-db.mmd`

4.10 insights_db (Insights)

From SRS v1.1: 8.4, 5.13 Insights and reports

Pre-added daily totals per platform, seller and product, so dashboards are fast and never touch other databases. Nothing here is edited by people; reports are stored snapshots.

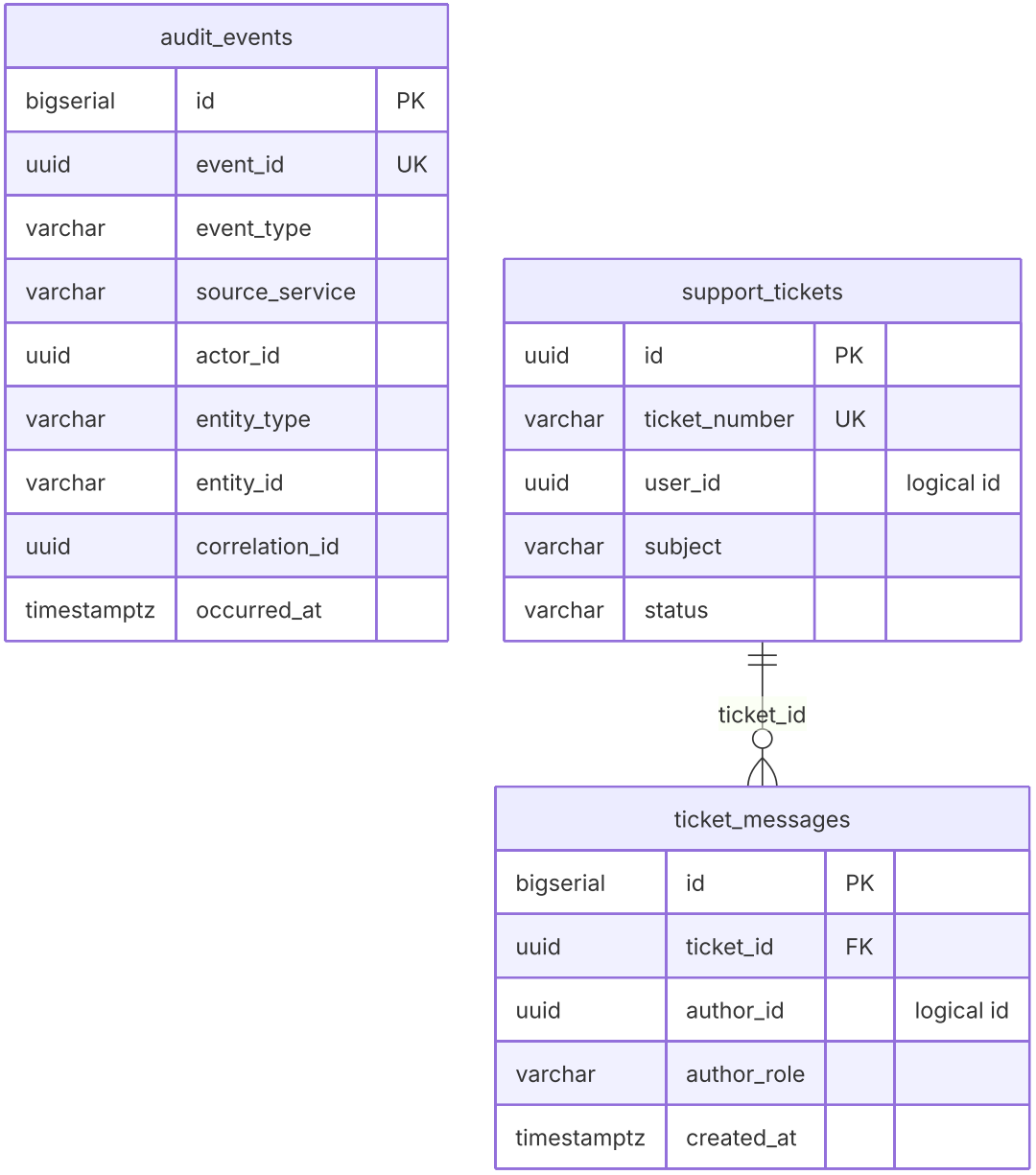


Source: `mermaid/20-erd-09-insights-db.mmd`

4.11 platform_db (Audit, support tickets)

From SRS v1.1: 8.4, FR-AUD-01 to 03, FR-SUP-01

`audit_events` is built and append-only. Support tickets and their messages are planned.



Source: `mermaid/20-erd-10-platform-db.mmd`

4.12 Search index (OpenSearch)

From SRS v1.1: 5.11, FR-SRCH-01 to 03

Search keeps no SQL tables. This is the shape of one document in the products index.

products_index		
keyword	product_id	PK
text	title	
keyword	category_path	
keyword	brand	
keyword	seller_id	
scaled_float	min_price	
boolean	in_stock	
keyword	status	

Source: `mermaid/20-erd-11-search-index.mmd`

4.13 Every table and every column

The full schema, database by database. **Bold** column names (● in the ERD column) are the ones drawn in the diagrams. Null "yes" means the column may be empty. FK is a real foreign key inside the same database; "ref" is a logical id owned by another service. Types are PostgreSQL types.

Shared tables (in every service that publishes or consumes events)

`outbox_events` Built

Events waiting to be published. Written in the same transaction as the business change, then sent to Kafka by the outbox relay (ADR 0002).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	also the event id
`aggregate_type`	`varchar(40)`				e.g. USER, SELLER
`aggregate_id`	`varchar(64)`			●	Kafka message key, keeps one entity's events in order
`event_type`	`varchar(80)`			●	
`topic`	`varchar(120)`			●	
`envelope`	`jsonb`				the full EventEnvelope (SRS 9.1)
`status`	`varchar(16)`			●	PENDING, PUBLISHED, FAILED
`attempts`	`int`				default 0
`last_error`	`varchar(1000)`	yes			
`created_at`	`timestamptz`			●	
`published_at`	`timestamptz`	yes			

Partial index on created_at WHERE status = 'PENDING' so the relay finds unsent rows fast.

`processed_events` Built

Event ids a consumer has already handled, so a redelivered event is skipped (idempotent consumer, SRS 9.4).

Column	Type	Null	Key	ERD	Notes
`consumer`	`varchar(80)`		PK	●	listener name
`event_id`	`uuid`		PK	●	
`processed_at`	`timestamptz`				default now()

identity_db · Identity service

Users, roles, MFA, sessions, sellers and automatic seller verification. Also has the shared `outbox_events` table.

`users` Built

Every person who can log in.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
name	varchar(120)			●	
email	varchar(254)		UK	●	unique on LOWER(email) WHERE deleted_at IS NULL
phone	varchar(20)			●	
password_hash	varchar(100)				BCrypt (FR-AUTH-02)
status	varchar(32)			●	PENDING_VERIFICATION, ACTIVE, LOCKED, SUSPENDED, DEACTIVATED
failed_login_attempts	int				default 0
locked_until	timestamptz	yes			
created_at	timestamptz				
updated_at	timestamptz				
version	bigint				optimistic locking, default 0
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

`roles` Built

The role catalogue.

Column	Type	Null	Key	ERD	Notes
name	varchar(32)		PK	●	CUSTOMER, SELLER, ADMIN
description	varchar(200)				

V1 also seeded a SUPPORT row. SRS v1.1 removed that role, so a later migration should delete it.

`user_roles` Built

Which roles each user holds (multi-role users, FR-AUTH-05).

Column	Type	Null	Key	ERD	Notes
user_id	uuid		PK, FK → users.id	●	
role	varchar(32)		PK, FK → roles.name	●	
granted_at	timestamptz				default now()

addresses Built

Saved delivery addresses. Orders copy them as snapshots.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid		FK → users.id	●	
label	varchar(40)	yes			
recipient	varchar(120)				
phone	varchar(20)				
line1	varchar(200)				
line2	varchar(200)	yes			
city	varchar(100)			●	
state	varchar(100)			●	decides CGST+SGST or IGST (FR-TAX-02)
postal_code	varchar(12)			●	
country	varchar(2)				default 'IN'
is_default	boolean				default false
created_at	timestamptz				
updated_at	timestamptz				
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

phone_verifications Planned

One-time SMS codes that verify a phone before the first order (FR-AUTH-09).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid		FK → users.id	●	
phone	varchar(20)			●	
code_hash	varchar(64)				SHA-256, the code itself is never stored
attempts	int				default 0
expires_at	timestamptz			●	
verified_at	timestamptz	yes		●	
created_at	timestamptz				

user_mfa Built

Authenticator-app (TOTP) enrolment per user (FR-AUTH-10, ADR 0004).

Column	Type	Null	Key	ERD	Notes
user_id	uuid		PK, FK → users.id	●	
totp_secret_encrypted	varchar(200)				AES-GCM
status	varchar(16)			●	PENDING, ACTIVE
last_used_step	bigint				blocks replay of the same 30-second code
created_at	timestamptz				
activated_at	timestamptz	yes			
version	bigint				optimistic locking, default 0

mfa_recovery_codes Built

One-time codes for a lost phone (named recovery_codes in SRS 8.4).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid		FK → users.id	●	
code_hash	varchar(64)		UK		
created_at	timestamptz				
used_at	timestamptz	yes		●	

`mfa_challenges` Built

Issued after a correct password when MFA is on; exchanged for tokens with a valid code.

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`user_id`	`uuid`		FK → users.id	●	
`token_hash`	`varchar(64)`		UK		
`attempts`	`int`				default 0, max 5
`created_at`	`timestamptz`				
`expires_at`	`timestamptz`			●	
`consumed_at`	`timestamptz`	yes			

`refresh_tokens` Built

Hashed refresh tokens. A family is one login's rotation chain (FR-AUTH-04, ADR 0003).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`user_id`	`uuid`		FK → users.id	●	
`family_id`	`uuid`			●	reuse of a rotated token revokes the whole family
`token_hash`	`varchar(64)`		UK		
`expires_at`	`timestamptz`			●	
`created_at`	`timestamptz`				
`revoked_at`	`timestamptz`	yes		●	
`revoke_reason`	`varchar(40)`	yes			
`replaced_by_id`	`uuid`	yes			
`mfa_verified`	`boolean`				refreshed tokens keep the strength of the login

`sellers` Built

A seller application and, once approved, the seller account (one per user).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`user_id`	`uuid`		UK, FK → users.id	●	
`business_name`	`varchar(160)`			●	
`business_email`	`varchar(254)`				
`business_phone`	`varchar(20)`				
`tax_id`	`varchar(20)`	yes		●	GSTIN
`pickup_address`	`varchar(500)`				
`status`	`varchar(32)`			●	DRAFT, PENDING_VERIFICATION, UNDER_REVIEW, APPROVED, REJECTED, SUSPENDED, CLOSED
`bank_ifsc`	`varchar(11)`	yes			
`bank_account_last4`	`varchar(4)`	yes			only the last four digits are readable
`kyc_inputs_enc`	`text`	yes			AES-GCM: account number, PAN, date (SRS 8.2)
`created_at`	`timestamptz`				
`updated_at`	`timestamptz`				
`version`	`bigint`				optimistic locking, default 0

`seller_documents` Built

KYC files in object storage; the row holds the key, never the file (FR-SELLER-10, ADR 0005).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`seller_id`	`uuid`		FK → sellers.id	●	
`document_type`	`varchar(40)`			●	
`storage_key`	`varchar(500)`		UK	●	
`content_type`	`varchar(100)`				
`size_bytes`	`bigint`				
`status`	`varchar(20)`			●	PENDING_UPLOAD, UPLOADED, INVALID, EXPIRED
`original_file_name`	`varchar(200)`	yes			
`created_at`	`timestamptz`				
`uploaded_at`	`timestamptz`	yes			

`seller_verifications` Built

One automatic verification attempt (ADR 0007).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`seller_id`	`uuid`		FK → sellers.id	●	
`status`	`varchar(24)`			●	AWAITING_IDENTITY, READY, RUNNING, PASSED, NEEDS_REVIEW, SUPERSEDED
`provider`	`varchar(20)`				mock, sandbox
`identity_session_id`	`varchar(100)`	yes			DigiLocker session
`attempts`	`int`			●	
`next_attempt_at`	`timestamptz`	yes			when the background worker retries
`locked_until`	`timestamptz`	yes			worker lease
`outcome_summary`	`varchar(500)`	yes			
`created_at`	`timestamptz`				
`completed_at`	`timestamptz`	yes			
`version`	`bigint`				optimistic locking, default 0

`seller_verification_checks` Built

Result of each check: GSTIN, PAN, bank account, identity. Append-only.

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`verification_id`	`uuid`		FK → seller_verifications.id	●	
`check_type`	`varchar(20)`			●	GSTIN, PAN, BANK_ACCOUNT, IDENTITY
`outcome`	`varchar(16)`			●	PASS, FAIL, MISMATCH, MISSING, ERROR
`detail`	`varchar(500)`				
`registry_name`	`varchar(200)`	yes			name the registry returned, for the admin
`match_score`	`numeric(4,3)`	yes			
`provider_ref`	`varchar(100)`	yes			
`checked_at`	`timestamptz`				

A trigger rejects UPDATE and DELETE.

`seller_status_history` Built

Every seller status change, never edited (FR-SELLER-03).

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`seller_id`	`uuid`		FK → sellers.id	●	
`from_status`	`varchar(32)`	yes			
`to_status`	`varchar(32)`			●	
`reason`	`varchar(500)`				
`actor_id`	`uuid`			●	user, or the SYSTEM id
`correlation_id`	`uuid`	yes			
`changed_at`	`timestamptz`			●	

A trigger rejects UPDATE and DELETE.

catalog_db · Catalog service

Categories, brands, products, variants, images, prices and automatic listing checks. Also has the shared `outbox_events`, `processed_events` tables.

verified_seller_projection Planned

Local copy of approved sellers, built from identity-events (FR-SELLER-04).

Column	Type	Null	Key	ERD	Notes
seller_id	uuid		PK, ref → identity.sellers	●	
business_name	varchar(160)			●	
gstin	varchar(15)			●	
registered_address	varchar(500)				shown on product pages (FR-CAT-12)
state_code	varchar(2)			●	
status	varchar(32)			●	APPROVED, SUSPENDED
last_event_id	uuid				
updated_at	timestamptz				

categories Planned

Category tree with the return window and commission rate (FR-CAT-01, FR-RET-02).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
parent_id	uuid	yes	FK → categories.id	●	
name	varchar(120)			●	
slug	varchar(140)		UK		
returnable	boolean			●	
return_window_days	int	yes		●	null when not returnable
commission_rate	numeric(5,4)			●	e.g. 0.0800
commission_effective_from	date				
is_active	boolean				
created_at	timestamptz				
updated_at	timestamptz				
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

brands Planned

Brands, managed by admin.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
name	varchar(120)		UK	●	
slug	varchar(140)		UK		
logo_storage_key	varchar(500)	yes			
status	varchar(16)			●	ACTIVE, RESTRICTED
created_at	timestamptz				
updated_at	timestamptz				
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

products Planned

A listing owned by one seller.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
seller_id	uuid		ref → identity.sellers	●	
category_id	uuid		FK → categories.id	●	
brand_id	uuid	yes	FK → brands.id	●	
title	varchar(200)			●	
slug	varchar(220)		UK		
description	text				
status	varchar(24)			●	DRAFT, PENDING_CHECKS, PENDING_APPROVAL, ACTIVE, INACTIVE, OUT_OF_STOCK, REJECTED, SUSPENDED, DELETED
hsn_code	varchar(8)			●	FR-CAT-11
gst_rate	numeric(5,4)			●	
country_of_origin	varchar(2)				FR-CAT-12
manufacturer_details	varchar(500)	yes			
is_packaged	boolean				MRP and net quantity required when true
trust_score	numeric(5,2)	yes			copied from trust-events
rejection_reason	varchar(500)	yes			
published_at	timestamptz	yes			
created_at	timestamptz				
updated_at	timestamptz				
version	bigint				optimistic locking, default 0

Column	Type	Null	Key	ERD	Notes
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

product_variants Planned

A buyable SKU: size, colour and so on.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
product_id	uuid		FK → products.id	●	
sku	varchar(64)		UK	●	unique WHERE deleted_at IS NULL (FR-CAT-04)
attributes	jsonb			●	{"size": "M", "colour": "blue"}
price	numeric(12,2)			●	
mrp	numeric(12,2)	yes		●	
net_quantity	numeric(10,3)	yes			
net_quantity_unit	varchar(10)	yes			
weight_grams	int				for shipping
is_active	boolean				
created_at	timestamptz				
updated_at	timestamptz				
version	bigint				optimistic locking, default 0
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

product_images Planned

Image metadata; the file lives in object storage (FR-CAT-05).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
product_id	uuid		FK → products.id	●	
variant_id	uuid	yes	FK → product_variants.id		
storage_key	varchar(500)			●	
width	int				
height	int				
sort_order	int				
alt_text	varchar(200)	yes			
created_at	timestamptz				
deleted_at	timestamptz	yes			

price_history Planned

Every price change, never edited (FR-CAT-07).

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
variant_id	uuid		FK → product_variants.id	●	
old_price	numeric(12,2)	yes			
new_price	numeric(12,2)			●	
old_mrp	numeric(12,2)	yes			
new_mrp	numeric(12,2)	yes			
changed_by	uuid				
changed_at	timestamptz			●	

`listing_checks` Planned

Result of each automatic listing rule (FR-CAT-10).

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`product_id`	`uuid`		FK → products.id	●	
`attempt`	`int`				
`rule`	`varchar(40)`			●	BANNED_ITEM, BANNED_WORDS, REQUIRED_DETAILS, IMAGE_QUALITY, CATEGORY_FIT, BRAND_RULE
`outcome`	`varchar(10)`			●	PASS, FAIL, UNSURE
`detail`	`varchar(500)`				
`checked_by`	`varchar(20)`				SYSTEM, AI_SERVICE
`checked_at`	`timestamptz`				

commerce_db · Commerce service

Inventory and reservations, cart, wishlist, orders split per seller, and dispatch deadlines. Also has the shared `outbox_events`, `processed_events` tables.

product_projection Planned

Local copy of sellable variants, built from product-events.

Column	Type	Null	Key	ERD	Notes
variant_id	uuid		PK, ref → catalog.product_variants	●	
product_id	uuid		ref → catalog.products	●	
seller_id	uuid		ref → identity.sellers	●	
category_id	uuid		ref → catalog.categories		
sku	varchar(64)				
title	varchar(200)				
attributes	jsonb				
price	numeric(12,2)			●	
mrp	numeric(12,2)	yes			
hsn_code	varchar(8)				
gst_rate	numeric(5,4)				
commission_rate	numeric(5,4)				
return_window_days	int	yes			
status	varchar(24)			●	
last_event_id	uuid				
updated_at	timestamptz				

inventory Planned

Stock per variant. available = on_hand - reserved (SRS 8.6).

Column	Type	Null	Key	ERD	Notes
variant_id	uuid		PK, FK → product_projection.variant_id	●	
seller_id	uuid		ref → identity.sellers	●	
on_hand_quantity	int			●	
reserved_quantity	int			●	
low_stock_threshold	int	yes			FR-INV-06
updated_at	timestamptz				
version	bigint				optimistic locking, default 0

CHECK (on_hand_quantity >= reserved_quantity) stops overselling (FR-INV-05).

stock_reservations Planned

Stock held for an order during checkout, with an expiry (FR-INV-04).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
variant_id	uuid		FK → inventory.variant_id	●	
order_id	uuid		FK → orders.id	●	
quantity	int			●	
status	varchar(16)			●	ACTIVE, CONFIRMED, RELEASED, EXPIRED, CANCELLED
expires_at	timestamptz			●	
created_at	timestamptz				
updated_at	timestamptz				

Partial index on (status, expires_at) WHERE status = 'ACTIVE' for the expiry job.

stock_movements Planned

Every stock change, never edited (FR-INV-03).

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
variant_id	uuid		FK → inventory.variant_id	●	
movement_type	varchar(16)			●	ADD, ADJUST, RESERVE, RELEASE, CONFIRM, RESTOCK
quantity_delta	int			●	
on_hand_after	int				
reserved_after	int				
reference_type	varchar(30)	yes			ORDER, RETURN, MANUAL
reference_id	uuid	yes			
actor_id	uuid				
created_at	timestamptz			●	

carts Planned

One open cart per customer.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid		UK, ref → identity.users	●	
created_at	timestamptz				
updated_at	timestamptz				

cart_items Planned

Lines in a cart. Checkout re-reads price and stock (FR-CART-02).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
cart_id	uuid		FK → carts.id	●	
variant_id	uuid		FK → product_projection.variant_id	●	unique per cart
quantity	int			●	
price_at_add	numeric(12,2)				display only, never charged
added_at	timestamptz				
updated_at	timestamptz				

wishlists Planned

One wishlist per customer.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid		UK, ref → identity.users	●	
created_at	timestamptz				

wishlist_items Planned, not in SRS 8.4

Lines in a wishlist; the price at add time drives the price-drop alert (FR-WISH-02).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
wishlist_id	uuid		FK → wishlists.id	●	
product_id	uuid		ref → catalog.products	●	
variant_id	uuid	yes	ref → catalog.product_variants		
price_at_add	numeric(12,2)			●	
added_at	timestamptz				

Not named in SRS 8.4, which lists only wishlists; a wishlist needs its items.

orders Planned

The customer's order, with address snapshots (SRS 8.7).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
order_number	varchar(20)		UK	●	human-readable
user_id	uuid		ref → identity.users	●	
status	varchar(24)			●	CREATED … REFUNDED (Appendix A)
subtotal_amount	numeric(12,2)				
tax_amount	numeric(12,2)				
shipping_amount	numeric(12,2)				
total_amount	numeric(12,2)			●	
currency	varchar(3)				INR
shipping_address	jsonb				snapshot
billing_address	jsonb				snapshot
buyer_state_code	varchar(2)				
placed_at	timestamptz				
created_at	timestamptz				
updated_at	timestamptz				
version	bigint				optimistic locking, default 0

Index on (user_id, created_at DESC) for 'my orders'.

seller_orders Planned

The part of an order one seller fulfils (FR-ORD-02).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
order_id	uuid		FK → orders.id	●	
seller_id	uuid		ref → identity.sellers	●	
status	varchar(24)			●	CONFIRMED, PACKED, SHIPPED, DELIVERED, CANCELLED, RETURNED_TO_ORIGIN
subtotal_amount	numeric(12,2)				
tax_amount	numeric(12,2)				
total_amount	numeric(12,2)			●	
dispatch_deadline_at	timestamptz	yes		●	FR-SELLER-08
confirmed_at	timestamptz	yes			
packed_at	timestamptz	yes			
shipped_at	timestamptz	yes			
delivered_at	timestamptz	yes			
return_window_ends_at	timestamptz	yes			
cancelled_at	timestamptz	yes			
cancel_reason	varchar(200)	yes			
created_at	timestamptz				
updated_at	timestamptz				
version	bigint				optimistic locking, default 0

`order_items` Planned

One line, with immutable product, price and tax snapshots (FR-ORD-03).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
order_id	uuid		FK → orders.id	●	
seller_order_id	uuid		FK → seller_orders.id	●	
variant_id	uuid		ref → catalog.product_variants	●	
product_id	uuid		ref → catalog.products		
sku	varchar(64)				
product_title	varchar(200)				
variant_attributes	jsonb				
quantity	int			●	
unit_price	numeric(12,2)			●	
mrp	numeric(12,2)	yes			
hsn_code	varchar(8)				
gst_rate	numeric(5,4)				
tax_amount	numeric(12,2)				
line_total	numeric(12,2)				
commission_rate	numeric(5,4)				snapshot so later rate changes do not alter this order
return_window_days	int	yes			
status	varchar(24)			●	ORDERED, CANCELLED, RETURN_REQUESTED, RETURNED
created_at	timestamptz				

`order_status_history` Planned

Every order and seller-order status change, never edited (FR-ORD-04).

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
order_id	uuid		FK → orders.id	●	
seller_order_id	uuid	yes	FK → seller_orders.id		
from_status	varchar(24)	yes			
to_status	varchar(24)			●	
reason	varchar(500)	yes			
actor_id	uuid				
correlation_id	uuid	yes			
changed_at	timestamptz			●	

`idempotency_keys` Planned

Makes checkout safe to retry: the same key returns the same order (FR-ORD-05).

Column	Type	Null	Key	ERD	Notes
user_id	uuid		PK, ref → identity.users	●	
idempotency_key	varchar(64)		PK	●	
request_hash	varchar(64)				same key with a different body is rejected
order_id	uuid	yes	FK → orders.id	●	
response_status	int	yes			
response_body	jsonb	yes			
created_at	timestamptz				
expires_at	timestamptz				

payment_db · Payment and settlement service

Payments and refunds, the double-entry ledger, settlements, payouts and GST invoices. Also has the shared `outbox_events`, `processed_events` tables.

`payments` Planned

One payment per order (FR-PAY-01).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`order_id`	`uuid`		UK, ref → commerce.orders	●	
`user_id`	`uuid`		ref → identity.users	●	
`amount`	`numeric(12,2)`			●	
`currency`	`varchar(3)`				
`status`	`varchar(24)`			●	CREATED, PENDING, AUTHORIZED, CAPTURED, FAILED, CANCELLED, PARTIALLY_REFUNDED, REFUNDED
`provider`	`varchar(20)`				
`provider_order_id`	`varchar(100)`	yes			
`created_at`	`timestamptz`				
`updated_at`	`timestamptz`				
`version`	`bigint`				optimistic locking, default 0

`payment_attempts` Planned

Each try at paying (FR-PAY-03). Card numbers are never stored (FR-PAY-08).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`payment_id`	`uuid`		FK → payments.id	●	
`provider_payment_id`	`varchar(100)`	yes			
`method`	`varchar(20)`			●	UPI, CARD, NETBANKING, WALLET
`status`	`varchar(16)`			●	STARTED, SUCCEEDED, FAILED
`amount`	`numeric(12,2)`				
`failure_code`	`varchar(40)`	yes			
`failure_reason`	`varchar(500)`	yes			
`created_at`	`timestamptz`				
`completed_at`	`timestamptz`	yes			

`payment_webhook_events` Planned

Raw provider webhooks, deduplicated (FR-PAY-04).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`provider`	`varchar(20)`		UK	●	unique with provider_event_id
`provider_event_id`	`varchar(100)`		UK	●	
`event_type`	`varchar(60)`				
`payload`	`jsonb`				
`signature_valid`	`boolean`				
`received_at`	`timestamptz`				
`processed_at`	`timestamptz`	yes		●	
`processing_error`	`varchar(1000)`	yes			

`refunds` Planned

Refunds to the original method, full or partial (FR-PAY-07).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`payment_id`	`uuid`		FK → payments.id	●	
`seller_order_id`	`uuid`		ref → commerce.seller_orders	●	
`order_item_id`	`uuid`	yes	ref → commerce.order_items		
`amount`	`numeric(12,2)`			●	
`reason`	`varchar(24)`			●	CANCELLED, RETURNED, RTO, DISPATCH_MISSED, PAYMENT_ERROR
`status`	`varchar(16)`			●	REQUESTED, PROCESSING, COMPLETED, FAILED
`provider_refund_id`	`varchar(100)`	yes			
`idempotency_key`	`varchar(64)`		UK		
`requested_at`	`timestamptz`				
`completed_at`	`timestamptz`	yes			

`ledger_accounts` Planned

Ledger accounts: platform-wide ones and one payable account per seller (SRS 8.8).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`code`	`varchar(30)`		UK	●	unique with owner_id: CUSTOMER_PAYMENTS, PLATFORM_COMMISSION, PLATFORM_FEES, TCS_WITHHELD, TDS_WITHHELD, SELLER_PAYABLE, REFUNDS, PAYOUTS
`owner_type`	`varchar(10)`			●	PLATFORM, SELLER
`owner_id`	`uuid`	yes	UK, ref → identity.sellers	●	
`normal_balance`	`varchar(6)`				DEBIT, CREDIT
`created_at`	`timestamptz`				

`ledger_entries` Planned

Double-entry lines. Each transaction's debits equal its credits; append-only (FR-SETL-01).

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`transaction_id`	`uuid`			●	groups the entries of one money movement
`account_id`	`uuid`		FK → ledger_accounts.id	●	
`debit_amount`	`numeric(12,2)`			●	one of debit or credit is zero
`credit_amount`	`numeric(12,2)`			●	
`reference_type`	`varchar(20)`			●	PAYMENT, REFUND, COMMISSION, FEE, TCS, TDS, PAYOUT
`reference_id`	`uuid`				
`description`	`varchar(200)`				
`reverses_entry_id`	`bigint`	yes	FK → ledger_entries.id		corrections are reversing entries
`created_at`	`timestamptz`				

A deferred constraint trigger checks SUM(debit) = SUM(credit) per transaction_id at commit; UPDATE and DELETE are rejected.

`rate_configs` Planned, not in SRS 8.4

Fee, TCS and TDS rates with an effective-from date (SRS 8.2, FR-TAX-03).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`rate_type`	`varchar(16)`			●	FIXED_FEE, TCS, TDS
`rate`	`numeric(6,5)`	yes			
`fixed_amount`	`numeric(12,2)`	yes			
`effective_from`	`date`			●	
`effective_to`	`date`	yes			
`created_by`	`uuid`				
`created_at`	`timestamptz`				

Not named in SRS 8.4. SRS 8.2 requires rates in configuration with an effective-from date; the commission rate comes from the category, so only fees and tax rates live here.

`settlements` Planned

One settlement per seller per cycle (FR-SETL-04).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`seller_id`	`uuid`		UK, ref → identity.sellers	●	unique with cycle_start
`cycle_start`	`date`		UK	●	
`cycle_end`	`date`			●	
`gross_amount`	`numeric(12,2)`				
`commission_amount`	`numeric(12,2)`				
`fee_amount`	`numeric(12,2)`				
`tcs_amount`	`numeric(12,2)`				
`tds_amount`	`numeric(12,2)`				
`refund_amount`	`numeric(12,2)`				
`net_payable`	`numeric(12,2)`			●	
`status`	`varchar(12)`			●	PENDING, PAYABLE, PAID, FAILED
`created_at`	`timestamptz`				
`paid_at`	`timestamptz`	yes			

`settlement_items` Planned

The seller orders inside a settlement, for the statement (FR-SETL-05).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`settlement_id`	`uuid`		FK → settlements.id	●	
`seller_order_id`	`uuid`		UK, ref → commerce.seller_orders	●	paid once only
`gross_amount`	`numeric(12,2)`				
`commission_amount`	`numeric(12,2)`				
`fee_amount`	`numeric(12,2)`				
`tcs_amount`	`numeric(12,2)`				
`tds_amount`	`numeric(12,2)`				
`refund_amount`	`numeric(12,2)`				
`net_amount`	`numeric(12,2)`			●	
`created_at`	`timestamptz`				

payouts Planned

Bank transfer of a settlement through the payout adapter.

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
settlement_id	uuid		UK, FK → settlements.id	●	
seller_id	uuid		ref → identity.sellers	●	
amount	numeric(12,2)			●	
status	varchar(16)			●	INITIATED, COMPLETED, FAILED
provider	varchar(20)				
provider_payout_id	varchar(100)	yes			
bank_account_last4	varchar(4)				
initiated_at	timestamptz				
completed_at	timestamptz	yes			
failure_reason	varchar(500)	yes			

invoices Planned

GST invoice in the seller's name per seller order (FR-TAX-01).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
invoice_number	varchar(30)		UK	●	
seller_order_id	uuid		UK, ref → commerce.seller_orders	●	
seller_id	uuid		ref → identity.sellers	●	
seller_gstin	varchar(15)				
seller_state_code	varchar(2)				
buyer_state_code	varchar(2)				
tax_type	varchar(6)			●	INTRA (CGST+SGST) or INTER (IGST), FR-TAX-02
taxable_amount	numeric(12,2)				
cgst_amount	numeric(12,2)				
sgst_amount	numeric(12,2)				
igst_amount	numeric(12,2)				
total_amount	numeric(12,2)			●	
storage_key	varchar(500)				the PDF
issued_at	timestamptz				

`invoice_items` Planned, not in SRS 8.4

Invoice lines, each with its own HSN rate (FR-TAX-02).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`invoice_id`	`uuid`		FK → invoices.id	●	
`order_item_id`	`uuid`		ref → commerce.order_items	●	
`description`	`varchar(200)`				
`hsn_code`	`varchar(8)`			●	
`quantity`	`int`				
`unit_price`	`numeric(12,2)`				
`taxable_amount`	`numeric(12,2)`				
`gst_rate`	`numeric(5,4)`			●	
`cgst_amount`	`numeric(12,2)`				
`sgst_amount`	`numeric(12,2)`				
`igst_amount`	`numeric(12,2)`				

fulfillment_db · Fulfillment service

Serviceability, shipments, courier tracking, failed deliveries (NDR/RTO), returns and reverse pickups. Also has the shared `outbox_events`, `processed_events` tables.

serviceable_pincodes Planned

Which pincodes each courier delivers to, and how fast (FR-SHIP-01).

Column	Type	Null	Key	ERD	Notes
pincode	varchar(6)		PK	●	
courier	varchar(20)		PK	●	
city	varchar(100)				
state_code	varchar(2)				
estimated_days	int			●	
is_active	boolean				
updated_at	timestamptz				

order_item_projection Planned, not in SRS 8.4

Order lines from commerce-events, to check return windows and quantities.

Column	Type	Null	Key	ERD	Notes
order_item_id	uuid		PK, ref → commerce.order_items	●	
seller_order_id	uuid		ref → commerce.seller_orders	●	
user_id	uuid		ref → identity.users	●	
variant_id	uuid		ref → catalog.product_variants		
quantity	int				
return_window_days	int	yes		●	null means non-returnable
last_event_id	uuid				

Not named in SRS 8.4; Fulfillment needs it to validate return requests on its own.

`shipments` Planned

One forward shipment per seller order (FR-SHIP-02).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`seller_order_id`	`uuid`		UK, ref → commerce.seller_orders	●	
`order_id`	`uuid`		ref → commerce.orders	●	
`seller_id`	`uuid`		ref → identity.sellers	●	
`status`	`varchar(24)`			●	BOOKED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED, DELIVERY_FAILED, RETURNED_TO_ORIGIN
`courier`	`varchar(20)`			●	
`awb_number`	`varchar(40)`		UK	●	
`label_storage_key`	`varchar(500)`				
`pickup_address`	`jsonb`				
`delivery_address`	`jsonb`				
`delivery_attempts`	`int`				
`max_delivery_attempts`	`int`				RTO after this (FR-SHIP-04)
`estimated_delivery_date`	`date`				
`booked_at`	`timestamptz`				
`picked_up_at`	`timestamptz`	yes			
`delivered_at`	`timestamptz`	yes			
`returned_to_origin_at`	`timestamptz`	yes			
`updated_at`	`timestamptz`				
`version`	`bigint`				optimistic locking,

Column	Type	Null	Key	ERD	Notes
					default 0

`shipment_events` Planned

Tracking timeline for a shipment, never edited.

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`shipment_id`	`uuid`		FK → shipments.id	●	
`status`	`varchar(24)`			●	
`location`	`varchar(120)`	yes			
`description`	`varchar(300)`	yes			
`occurred_at`	`timestamptz`			●	
`source`	`varchar(10)`				COURIER, SYSTEM
`courier_webhook_event_id`	`uuid`	yes	FK → courier_webhook_events.id		
`recorded_at`	`timestamptz`				

`courier_webhook_events` Planned

Raw signed courier webhooks, deduplicated (FR-SHIP-03).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`provider`	`varchar(20)`		UK	●	unique with provider_event_id
`provider_event_id`	`varchar(100)`		UK	●	
`awb_number`	`varchar(40)`				
`event_type`	`varchar(40)`				
`payload`	`jsonb`				
`signature_valid`	`boolean`				
`received_at`	`timestamptz`				
`processed_at`	`timestamptz`	yes		●	
`processing_error`	`varchar(1000)`	yes			

`return_requests` Planned

A customer's return of one order item (FR-RET-03).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`order_item_id`	`uuid`		FK → order_item_projection.order_item_id	●	
`seller_order_id`	`uuid`		ref → commerce.seller_orders	●	
`user_id`	`uuid`		ref → identity.users	●	
`quantity`	`int`				
`reason`	`varchar(40)`			●	DAMAGED, WRONG_ITEM, NOT_AS_DESCRIBED, SIZE_ISSUE, OTHER
`comment`	`varchar(1000)`	yes			
`photo_storage_keys`	`jsonb`	yes			
`status`	`varchar(20)`			●	REQUESTED, PICKUP_SCHEDULED, PICKED_UP, RECEIVED, REFUNDED, REJECTED
`resellable`	`boolean`	yes			set when the seller receives it
`rejection_reason`	`varchar(500)`	yes			
`requested_at`	`timestamptz`				
`received_at`	`timestamptz`	yes			
`updated_at`	`timestamptz`				
`version`	`bigint`				optimistic locking, default 0

`reverse_pickups` Planned

Courier pickup that brings a return back to the seller (FR-RET-04).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`return_request_id`	`uuid`		UK, FK → return_requests.id	●	
`courier`	`varchar(20)`				
`awb_number`	`varchar(40)`		UK	●	
`status`	`varchar(20)`			●	BOOKED, PICKED_UP, IN_TRANSIT, DELIVERED_TO_SELLER, FAILED
`scheduled_for`	`date`				
`picked_up_at`	`timestamptz`	yes			
`delivered_to_seller_at`	`timestamptz`	yes			
`updated_at`	`timestamptz`				

notification_db · Notification service

Email, SMS and in-app messages made from events, with retries and delivery history. Also has the shared `processed_events` table.

`user_contacts` Planned, not in SRS 8.4

Email and phone per user, from identity-events, so messages can be sent.

Column	Type	Null	Key	ERD	Notes
`user_id`	`uuid`		PK, ref → identity.users	●	
`email`	`varchar(254)`			●	
`phone`	`varchar(20)`			●	
`name`	`varchar(120)`				
`last_event_id`	`uuid`				
`updated_at`	`timestamptz`				

Not named in SRS 8.4; events carry ids, so Notification keeps its own contact copy.

`notifications` Planned

One message to one user on one channel (FR-NOT-01).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`user_id`	`uuid`		FK → user_contacts.user_id	●	
`channel`	`varchar(10)`			●	EMAIL, SMS, IN_APP
`template`	`varchar(60)`			●	
`subject`	`varchar(200)`	yes			
`body`	`text`				
`status`	`varchar(10)`			●	PENDING, SENT, FAILED
`source_event_id`	`uuid`		UK		unique with channel: one message per event per channel
`source_event_type`	`varchar(80)`				
`read_at`	`timestamptz`	yes			in-app only
`created_at`	`timestamptz`				
`sent_at`	`timestamptz`	yes			

`delivery_attempts` Planned

Each send attempt and its result, for retries.

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`notification_id`	`uuid`		FK → notifications.id	●	
`attempt_no`	`int`				
`provider`	`varchar(20)`				
`provider_message_id`	`varchar(100)`	yes			
`outcome`	`varchar(10)`			●	SENT, FAILED
`error`	`varchar(500)`	yes			
`attempted_at`	`timestamptz`				

`preferences` Planned

Which messages a user wants on which channel.

Column	Type	Null	Key	ERD	Notes
`user_id`	`uuid`		PK, FK → user_contacts.user_id	●	
`channel`	`varchar(10)`		PK	●	
`topic`	`varchar(30)`		PK	●	ORDER_UPDATES, PRICE_DROPS, PROMOTIONS
`enabled`	`boolean`				
`updated_at`	`timestamptz`				

Security codes (OTP, MFA) ignore preferences and are always sent.

trust_db · Trust service

Verified reviews, fake-review flags, product and seller trust scores with history. Also has the shared `outbox_events`, `processed_events` tables.

delivered_item_projection Planned

Delivered order items, the proof a reviewer bought the product (FR-REV-01).

Column	Type	Null	Key	ERD	Notes
order_item_id	uuid		PK, ref → commerce.order_items	●	
user_id	uuid		ref → identity.users	●	
product_id	uuid		ref → catalog.products	●	
seller_id	uuid		ref → identity.sellers	●	
variant_id	uuid		ref → catalog.product_variants		
delivered_at	timestamptz			●	
returned	boolean				
last_event_id	uuid				

reviews Planned

A verified buyer's review (FR-REV-01, FR-REV-02).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
order_item_id	uuid		UK, FK → delivered_item_projection.order_item_id	●	unique WHERE deleted_at IS NULL
user_id	uuid		ref → identity.users	●	
product_id	uuid		ref → catalog.products	●	
seller_id	uuid		ref → identity.sellers		
rating	smallint			●	1 to 5
title	varchar(150)	yes			
body	text	yes			
status	varchar(20)			●	PUBLISHED, PENDING_MODERATION, FLAGGED, HIDDEN, REMOVED
published_at	timestamptz	yes			
created_at	timestamptz				
updated_at	timestamptz				
deleted_at	timestamptz	yes			soft delete (SRS 8.3)
deleted_by	uuid	yes			
delete_reason	varchar(500)	yes			

review_flags Planned

A reason a review looks fake; admin decides (FR-REV-04).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
review_id	uuid		FK → reviews.id	●	
rule	varchar(20)			●	BURST, SELLER_LINKED, NEAR_DUPLICATE, USER_REPORT
detail	varchar(500)				
score	numeric(4,3)	yes			
status	varchar(10)			●	OPEN, UPHELD, DISMISSED
decided_by	uuid	yes			
decision_reason	varchar(500)	yes			
created_at	timestamptz				
decided_at	timestamptz	yes			

review_summaries Planned

AI summary of what reviewers praise and complain about (FR-INS-02).

Column	Type	Null	Key	ERD	Notes
product_id	uuid		PK, ref → catalog.products	●	
praise	text				
complaints	text				
review_count	int				
model_version	varchar(40)				
generated_at	timestamptz			●	

product_scores Planned

Current product trust score (FR-TRUST-01).

Column	Type	Null	Key	ERD	Notes
product_id	uuid		PK, ref → catalog.products	●	
score	numeric(5,2)			●	
avg_rating	numeric(3,2)				
review_count	int				
components	jsonb				the reasons behind the score
algorithm_version	varchar(20)				
updated_at	timestamptz				

seller_scores Planned

Current seller trust score with delivery and return performance (FR-TRUST-02, FR-SELLER-09).

Column	Type	Null	Key	ERD	Notes
seller_id	uuid		PK, ref → identity.sellers	●	
score	numeric(5,2)			●	
orders_count	int				
cancelled_count	int				
late_dispatch_count	int				
returned_count	int				
cancellation_rate	numeric(5,4)			●	
late_dispatch_rate	numeric(5,4)			●	
return_rate	numeric(5,4)			●	
avg_rating	numeric(3,2)				
components	jsonb				
flagged	boolean			●	crossed a threshold, admin reviews
algorithm_version	varchar(20)				
updated_at	timestamptz				

`score_history` Planned

Every score change with its reason components, never edited (FR-TRUST-03).

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`subject_type`	`varchar(10)`			●	PRODUCT, SELLER
`subject_id`	`uuid`			●	
`old_score`	`numeric(5,2)`	yes			
`new_score`	`numeric(5,2)`			●	
`components`	`jsonb`				
`trigger_event_id`	`uuid`	yes			
`recorded_at`	`timestamptz`			●	

recommendation_db · Recommendation service

User activity, ranking inputs, recommendation logs with reasons, and the AI call log. Also has the shared `outbox_events`, `processed_events` tables.

product_features — Planned, not in SRS 8.4

Ranking inputs per product, from product and trust events.

Column	Type	Null	Key	ERD	Notes
product_id	uuid		PK, ref → catalog.products	●	
category_id	uuid		ref → catalog.categories	●	
brand_id	uuid	yes	ref → catalog.brands		
min_price	numeric(12,2)			●	
trust_score	numeric(5,2)	yes			
avg_rating	numeric(3,2)	yes			
attributes	jsonb				
active	boolean			●	only active products can be recommended (FR-AI-02)
last_event_id	uuid				
updated_at	timestamptz				

Not named in SRS 8.4; content-based ranking (FR-REC-02) needs product attributes locally.

user_activity — Planned

Views, searches, cart, wishlist, purchase and review activity (FR-REC-01).

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
user_id	uuid	yes	ref → identity.users	●	null for anonymous sessions
session_id	varchar(64)				
activity_type	varchar(16)			●	VIEW, SEARCH, CART, WISHLIST, PURCHASE, REVIEW, CLICK
product_id	uuid	yes	FK → product_features.product_id	●	
query_text	varchar(300)	yes			
metadata	jsonb	yes			
occurred_at	timestamptz			●	

user_preferences — Planned

Learned preferences per user.

Column	Type	Null	Key	ERD	Notes
user_id	uuid		PK, ref → identity.users	●	
preferred_categories	jsonb				
preferred_brands	jsonb				
price_band_min	numeric(12,2)	yes			
price_band_max	numeric(12,2)	yes			
updated_at	timestamptz				

ai_requests Planned

Every AI call with its time and cost limit result (FR-AI-04).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid	yes	ref → identity.users		
request_type	varchar(20)			●	INTENT, COMPARE, EXPLAIN, REVIEW_SUMMARY, LISTING_CHECK
prompt_text	text				
response	jsonb	yes			
model	varchar(60)				
input_tokens	int				
output_tokens	int				
cost_usd	numeric(10,6)				
latency_ms	int			●	
status	varchar(10)			●	OK, TIMEOUT, ERROR, REJECTED
created_at	timestamptz				

recommendation_logs Planned

One recommendation request and the algorithm version used (FR-REC-03).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
user_id	uuid	yes	ref → identity.users	●	
context	varchar(16)			●	HOME, PRODUCT_PAGE, CART, AI_ASSISTANT
algorithm_version	varchar(20)			●	
request_params	jsonb				
ai_request_id	uuid	yes	FK → ai_requests.id		
created_at	timestamptz				

recommendation_items Planned

Each recommended product with its rank and reasons.

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
log_id	uuid		FK → recommendation_logs.id	●	
product_id	uuid		FK → product_features.product_id	●	
rank	int			●	
score	numeric(8,5)				
reasons	jsonb			●	e.g. ["matches your budget", "high trust score"]
clicked_at	timestamptz	yes			

product_similarity Planned

Precomputed similar products.

Column	Type	Null	Key	ERD	Notes
product_id	uuid		PK, FK → product_features.product_id	●	
similar_product_id	uuid		PK, FK → product_features.product_id	●	
score	numeric(6,5)			●	
method	varchar(20)				CONTENT, CO_PURCHASE
computed_at	timestamptz				

insights_db · Insights service

Read-only daily totals, alerts and report snapshots for the seller and admin dashboards (SRS 5.13). Also has the shared `processed_events` table.

daily_platform_totals Planned

Platform totals per day (FR-INS-04).

Column	Type	Null	Key	ERD	Notes
day	date		PK	●	
orders_count	int			●	
gmv	numeric(14,2)			●	gross merchandise value
net_revenue	numeric(14,2)				commission and fees
visitors_count	int				
conversion_rate	numeric(5,4)				
cancellations_count	int				
returns_count	int				
refunds_amount	numeric(14,2)				
payment_failures_count	int				
new_customers_count	int				
new_sellers_count	int				
updated_at	timestamptz				

daily_seller_totals Planned

Seller totals per day (FR-INS-01).

Column	Type	Null	Key	ERD	Notes
day	date		PK	●	
seller_id	uuid		PK, ref → identity.sellers	●	
orders_count	int			●	
revenue	numeric(14,2)			●	
units_sold	int				
cancellations_count	int				
late_dispatch_count	int				
returns_count	int				
refunds_amount	numeric(14,2)				
payouts_amount	numeric(14,2)				
updated_at	timestamptz				

daily_product_totals Planned

Product totals per day, for trends and top products (FR-INS-02).

Column	Type	Null	Key	ERD	Notes
day	date		PK	●	
product_id	uuid		PK, ref → catalog.products	●	
seller_id	uuid		ref → identity.sellers	●	
views_count	int				
add_to_cart_count	int				
units_sold	int			●	
revenue	numeric(14,2)			●	
returns_count	int				
updated_at	timestamptz				

`seller_action_items` Planned, not in SRS 8.4

What needs the seller's attention now: dispatch due, low stock, latest settlement (FR-INS-03).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`seller_id`	`uuid`		ref → identity.sellers	●	
`item_type`	`varchar(20)`			●	DISPATCH_DUE, LOW_STOCK, SETTLEMENT
`reference_id`	`uuid`			●	seller order, variant or settlement
`due_at`	`timestamptz`	yes			
`detail`	`jsonb`				
`open`	`boolean`			●	
`updated_at`	`timestamptz`				

Not named in SRS 8.4. SRS 3.5 says dashboards read only from Insights, so these live here too.

`alert_feed` Planned

Alerts and exceptions waiting for an admin decision (FR-INS-06).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`alert_type`	`varchar(24)`			●	FAKE_REVIEW, SELLER_FLAGGED, SELLER_REVIEW, LISTING_REVIEW, RISK, SUPPORT_TICKET
`severity`	`varchar(8)`				LOW, MEDIUM, HIGH
`subject_type`	`varchar(20)`			●	
`subject_id`	`uuid`			●	
`title`	`varchar(200)`				
`detail`	`jsonb`				
`status`	`varchar(10)`			●	OPEN, RESOLVED
`source_event_id`	`uuid`		UK		
`created_at`	`timestamptz`				
`resolved_at`	`timestamptz`	yes			

`report_runs` Planned

Generated reports; a run is a snapshot and is never edited (FR-INS-08).

Column	Type	Null	Key	ERD	Notes
`id`	`uuid`		PK	●	
`report_type`	`varchar(30)`			●	
`period_from`	`date`			●	
`period_to`	`date`			●	
`format`	`varchar(4)`			●	CSV, PDF
`status`	`varchar(10)`			●	QUEUED, RUNNING, DONE, FAILED
`storage_key`	`varchar(500)`	yes			
`row_count`	`int`	yes			
`checksum`	`varchar(64)`	yes			
`requested_by`	`uuid`		ref → identity.users		
`created_at`	`timestamptz`				
`completed_at`	`timestamptz`	yes			

platform_db · Audit (and support tickets) service

The append-only audit log of every event, and support tickets answered by admin. Also has the shared `processed_events` table.

audit_events Built

One row per domain event; append-only (FR-AUD-01 to 03, ADR 0006).

Column	Type	Null	Key	ERD	Notes
id	bigserial		PK	●	
event_id	uuid		UK	●	replays cannot duplicate
event_type	varchar(80)			●	
event_version	int				
source_service	varchar(60)			●	
actor_id	uuid	yes		●	
entity_type	varchar(40)			●	
entity_id	varchar(64)			●	
reason	varchar(1000)	yes			
correlation_id	uuid	yes		●	
occurred_at	timestamptz			●	
recorded_at	timestamptz				default now()
details	jsonb				

A trigger rejects UPDATE and DELETE. Indexes: (entity_type, entity_id, occurred_at), actor_id, occurred_at, correlation_id.

support_tickets Planned

A customer's support request, answered by admin (FR-SUP-01).

Column	Type	Null	Key	ERD	Notes
id	uuid		PK	●	
ticket_number	varchar(20)		UK	●	
user_id	uuid		ref → identity.users	●	
order_id	uuid	yes	ref → commerce.orders		
subject	varchar(200)			●	
category	varchar(30)				ORDER, PAYMENT, RETURN, ACCOUNT, OTHER
status	varchar(16)			●	OPEN, IN_PROGRESS, WAITING_ON_CUSTOMER, RESOLVED, CLOSED
priority	varchar(8)				
assigned_admin_id	uuid	yes	ref → identity.users		
created_at	timestamptz				
updated_at	timestamptz				
resolved_at	timestamptz	yes			

`ticket_messages` Planned

The conversation on a ticket, never edited.

Column	Type	Null	Key	ERD	Notes
`id`	`bigserial`		PK	●	
`ticket_id`	`uuid`		FK → support_tickets.id	●	
`author_id`	`uuid`		ref → identity.users	●	
`author_role`	`varchar(10)`			●	CUSTOMER, ADMIN
`body`	`text`				
`attachment_keys`	`jsonb`	yes			
`created_at`	`timestamptz`			●	

OpenSearch · Search service

products (OpenSearch index) Planned

One document per active product, built from product, stock and trust events (FR-SRCH-01).

Column	Type	Null	Key	ERD	Notes
product_id	keyword		PK	●	
title	text			●	analysed for keyword search
description	text				
category_path	keyword			●	e.g. electronics/phones
brand	keyword			●	
seller_id	keyword			●	
seller_name	keyword				
min_price	scaled_float			●	
max_price	scaled_float				
attributes	object				facets such as size and colour
in_stock	boolean			●	
avg_rating	float				
trust_score	float				
review_count	integer				
status	keyword			●	
published_at	date				
suggest	completion				autocomplete (FR-SRCH-03)

5. Request-flow diagrams

Sequence diagrams show one scenario in time order, top to bottom. Each column is a participant. Solid arrows are direct calls that wait for an answer; dashed arrows with an open head are Kafka events that nobody waits for. `alt` boxes show the different outcomes, `loop` a repeated step.

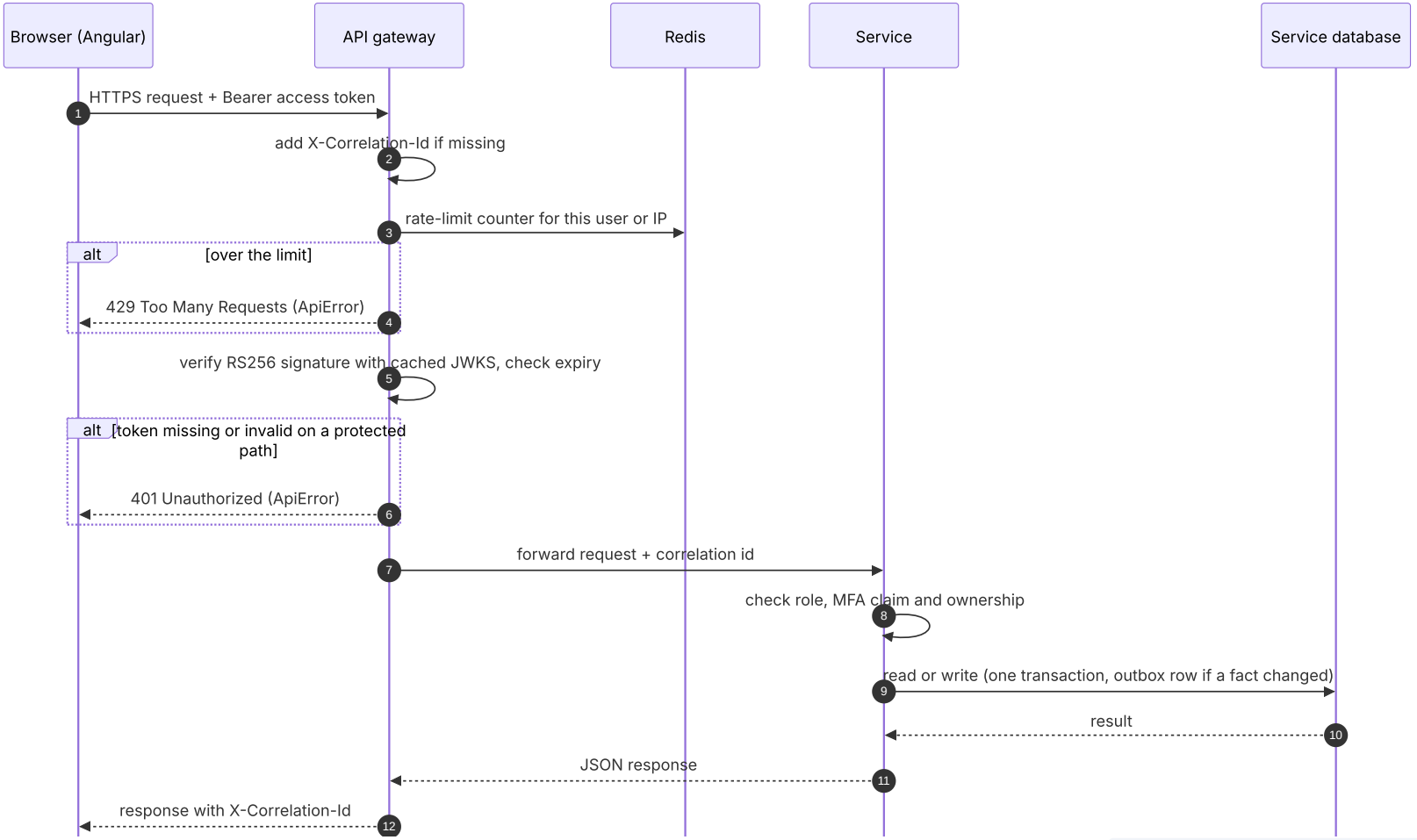
5.1 Any request through the gateway

From SRS v1.1: 12 Security, 12.1 Standard error response, 6.6 Observability

The path every API call takes, and where 429 and 401 errors come from.

Key ideas

- **Correlation id**: one id attached at the gateway and passed to every service and event, so the logs of one user action can be found together across services.
- **JWKS**: the gateway checks tokens with Identity's public keys, so it never needs the private key.

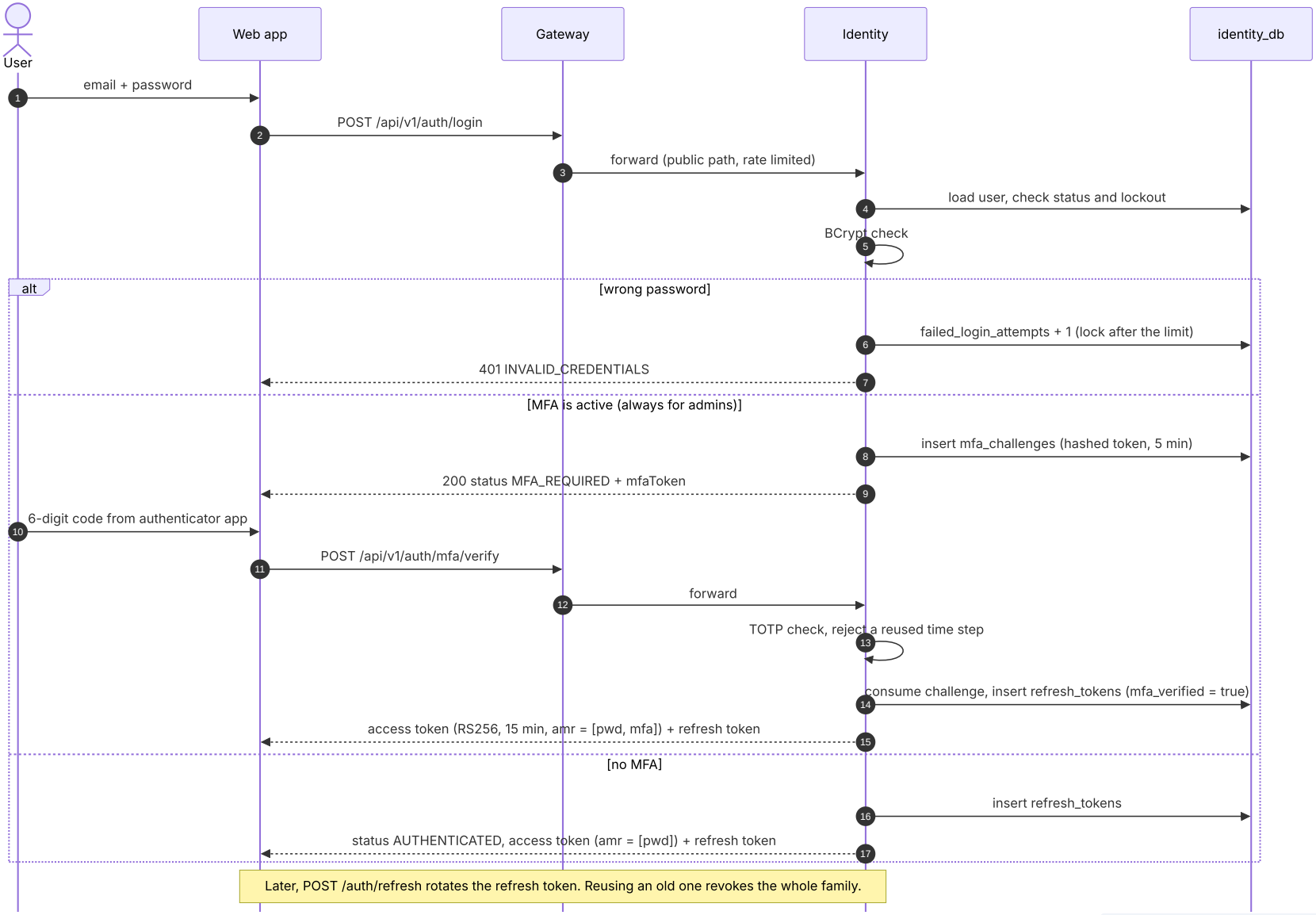


Source: `mermaid/30-request-through-gateway.mmd`

5.2 Login with MFA

From SRS v1.1: 5.1, ADR 0003, ADR 0004

Built in Phase 1. Admins always get the second step; other users only if they turned MFA on.



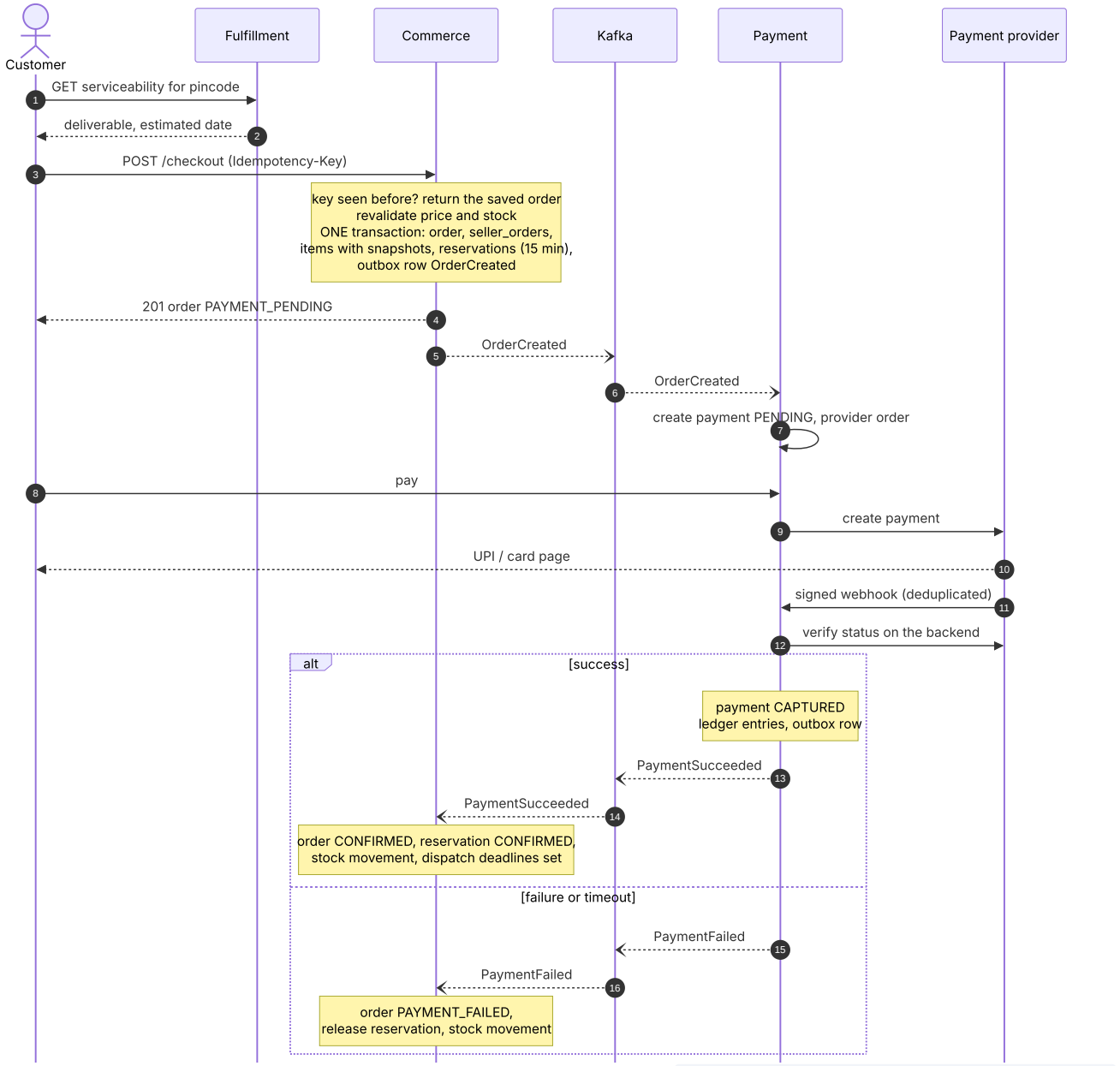
Source: `mermaid/31-login-with-mfa.mmd`

5.3 Checkout and payment saga

From SRS v1.1: 10.1, 5.4, 5.5, 5.8

The most important flow. Commerce creates the order and reserves stock in one transaction; Payment reacts to the event, and Commerce reacts to the payment result.

> **Key ideas**
>
> - **Saga**: a business process split across services, each step local and committed, with a compensating step if something later fails. Here the compensation for a failed payment is releasing the reserved stock. There is no distributed transaction.
> - **Idempotency key**: the browser sends the same key if it retries checkout, and gets the same order back instead of a second one.

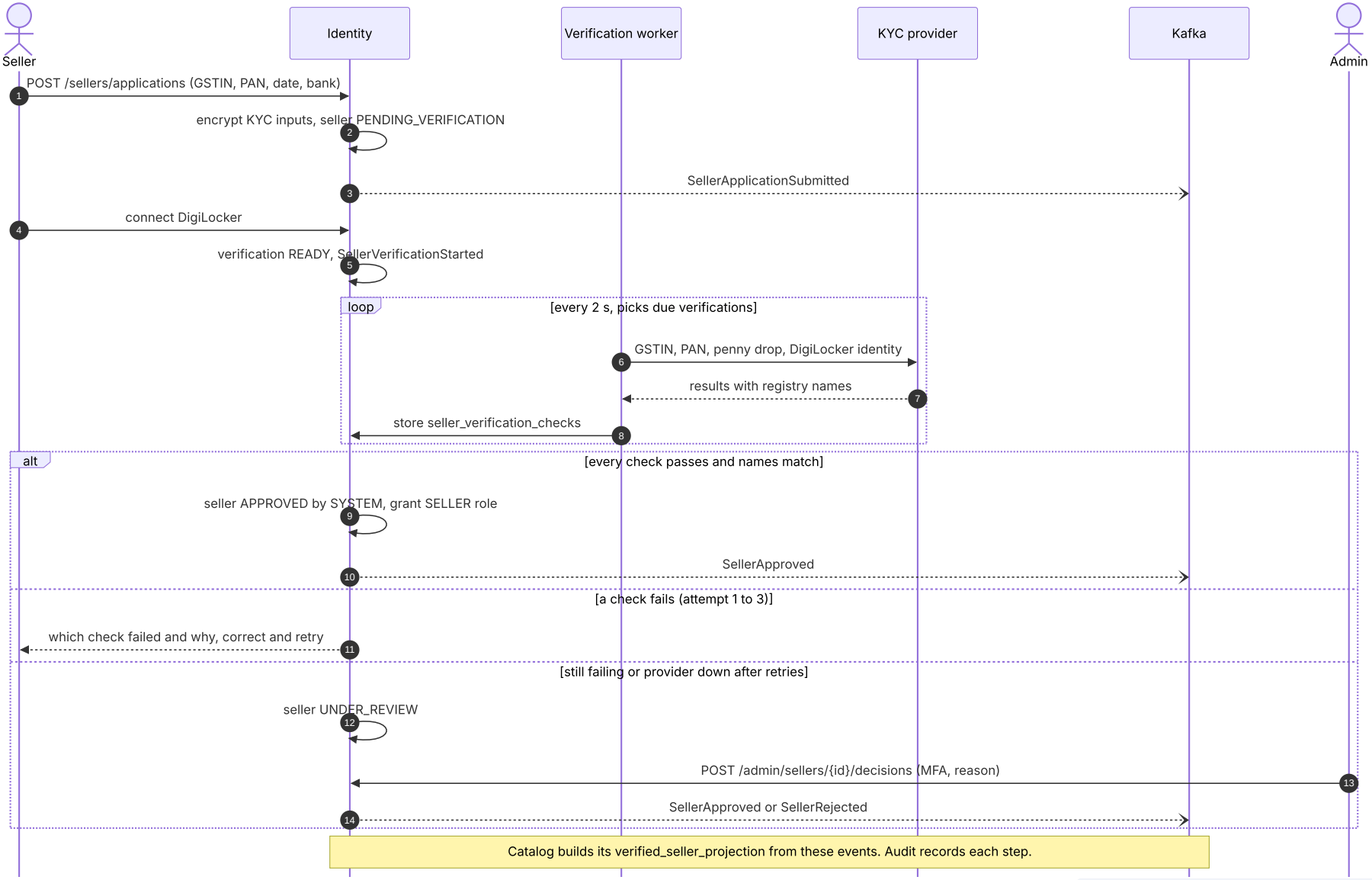


Source: `mermaid/32-checkout-and-payment.mmd`

5.4 Seller verification

From SRS v1.1: 10.2, FR-SELLER-02 and 07, ADR 0007

Built in Phase 1. Clean applications are approved by the system; the admin sees only exceptions.

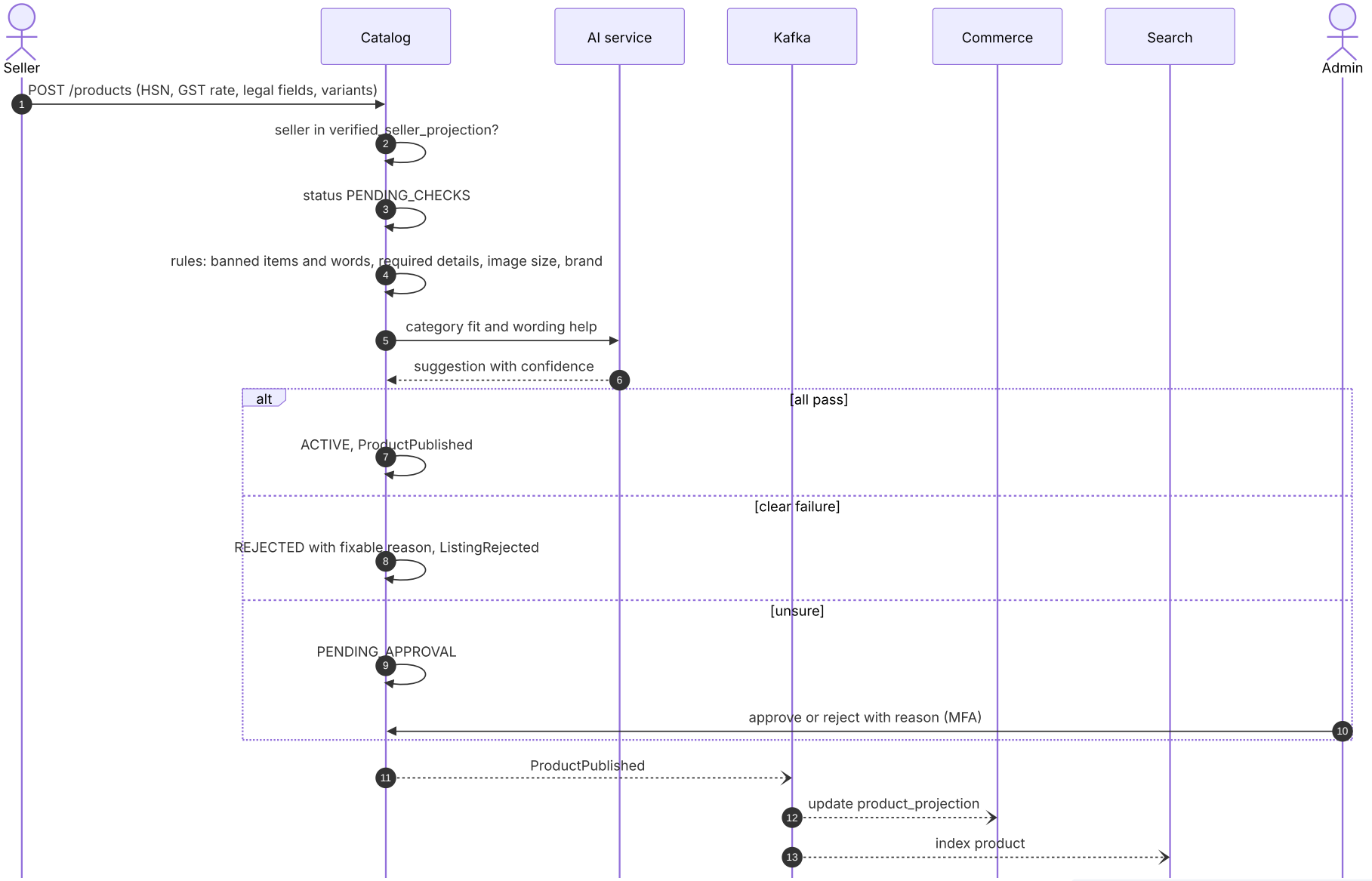


Source: `mermaid/33-seller-verification.mmd`

5.5 Product publication

From SRS v1.1: 10.3, FR-CAT-10

Automatic checks decide most listings; only doubtful ones reach an admin.

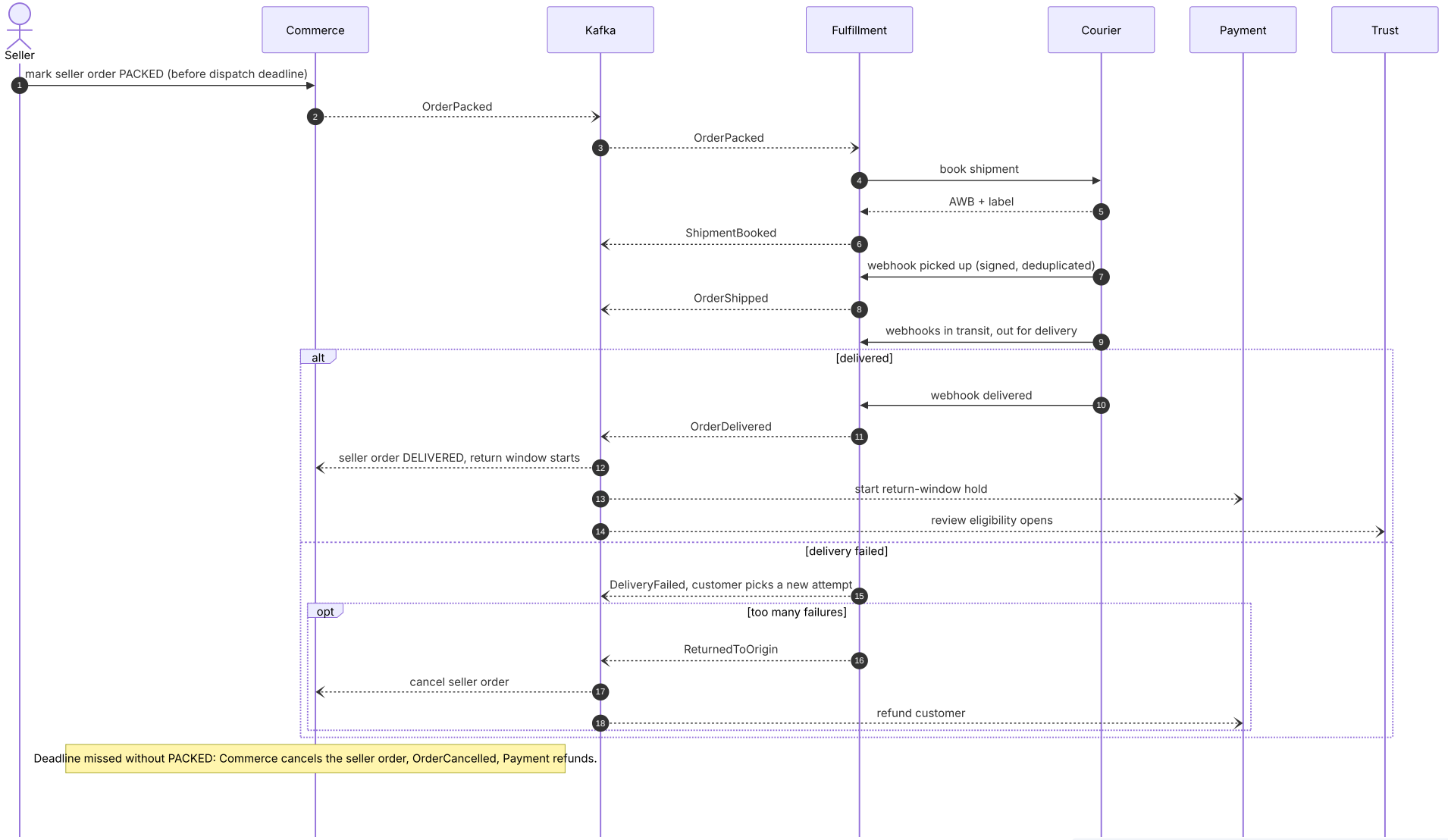


Source: `mermaid/34-product-publication.mmd`

5.6 Shipping and delivery

From SRS v1.1: 10.4, 5.6

One `OrderDelivered` event fans out to four services, each doing its own part.

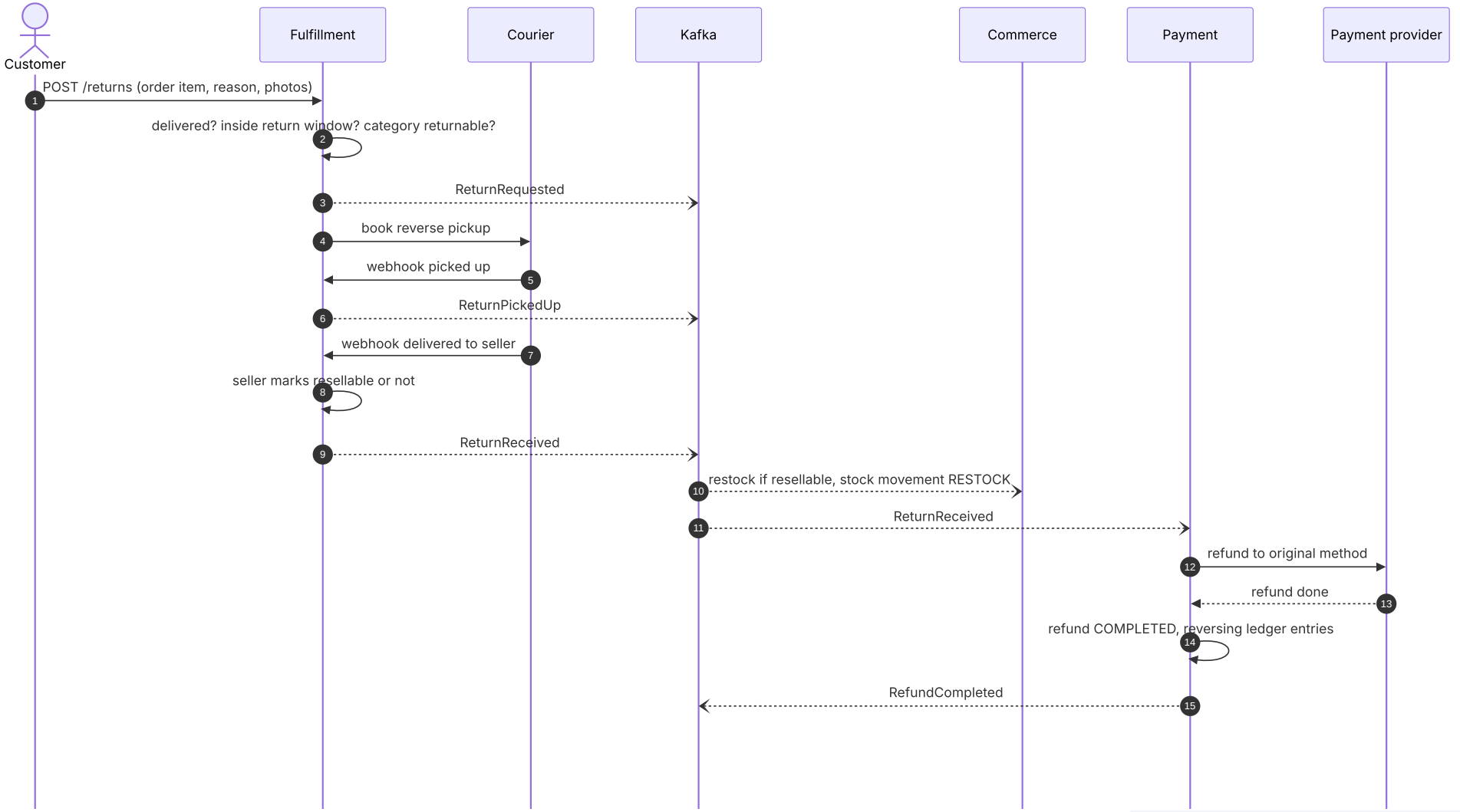


Source: `mermaid/35-shipping-and-delivery.mmd`

5.7 Return and refund

From SRS v1.1: 10.5, 5.7

Refunds are new reversing ledger entries; nothing already recorded is edited.



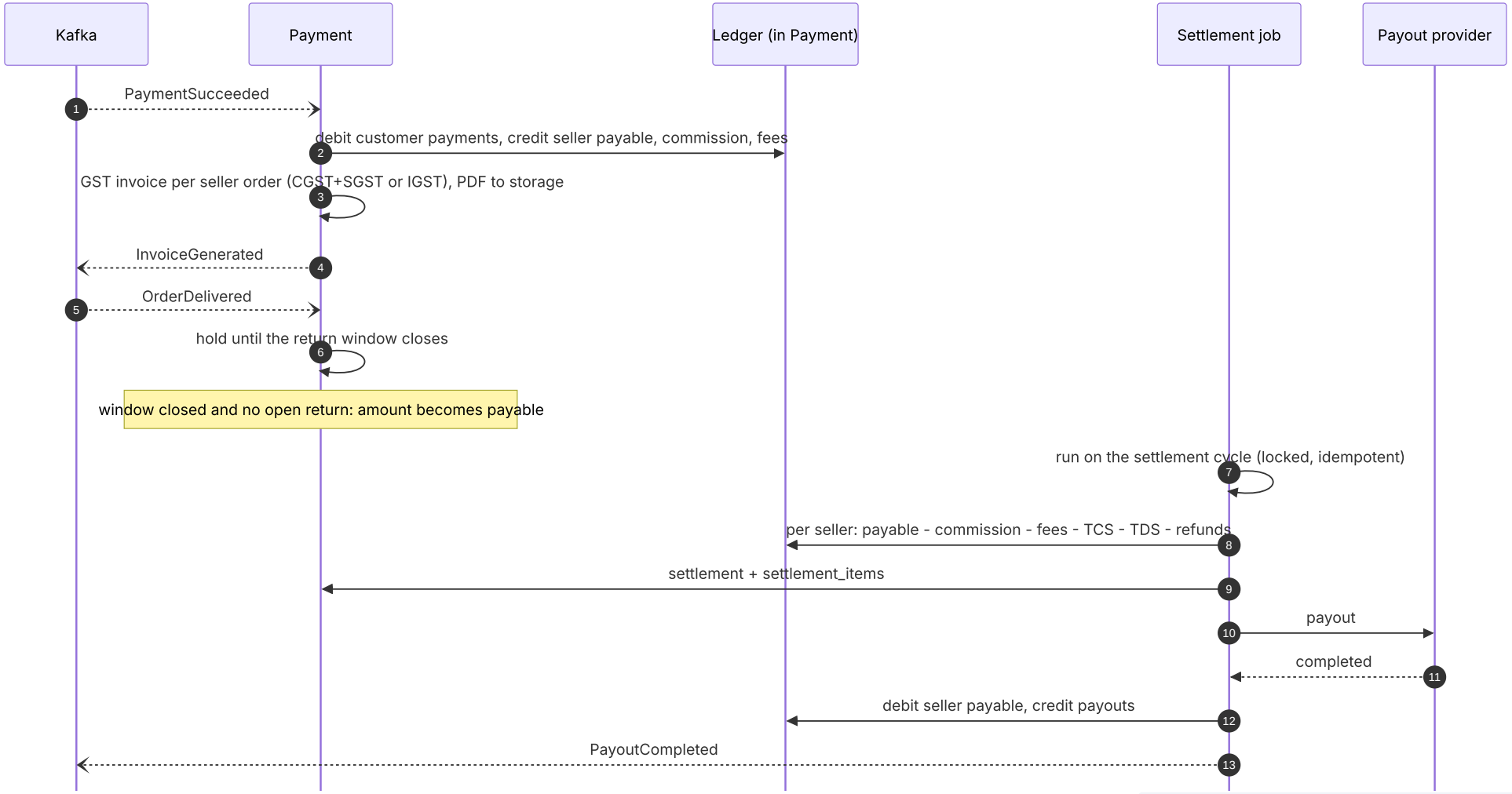
Source: `mermaid/36-return-and-refund.mmd`

5.8 Settlement and payout

From SRS v1.1: 10.6, 5.9, 8.8

Money is held until the return window closes, then settled per seller on a cycle.

> **Key ideas**
>
> - **Double-entry ledger**: every money movement is written as equal debits and credits across accounts, so the books always balance and any balance can be recomputed by adding entries.

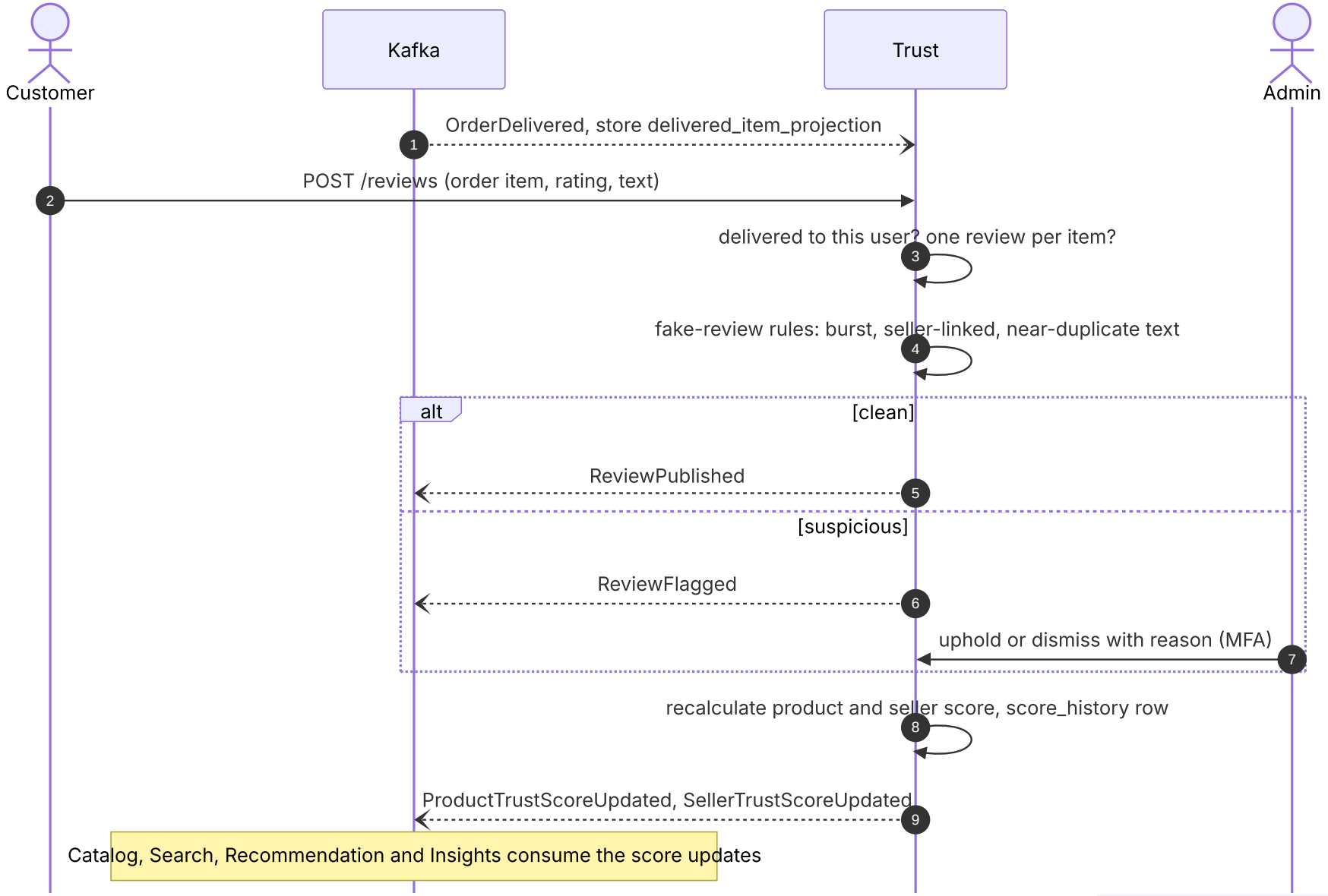


Source: `mermaid/37-settlement-and-payout.mmd`

5.9 Review and trust

From SRS v1.1: 10.7, 5.10

Only verified buyers can review, and suspicious reviews wait for an admin.

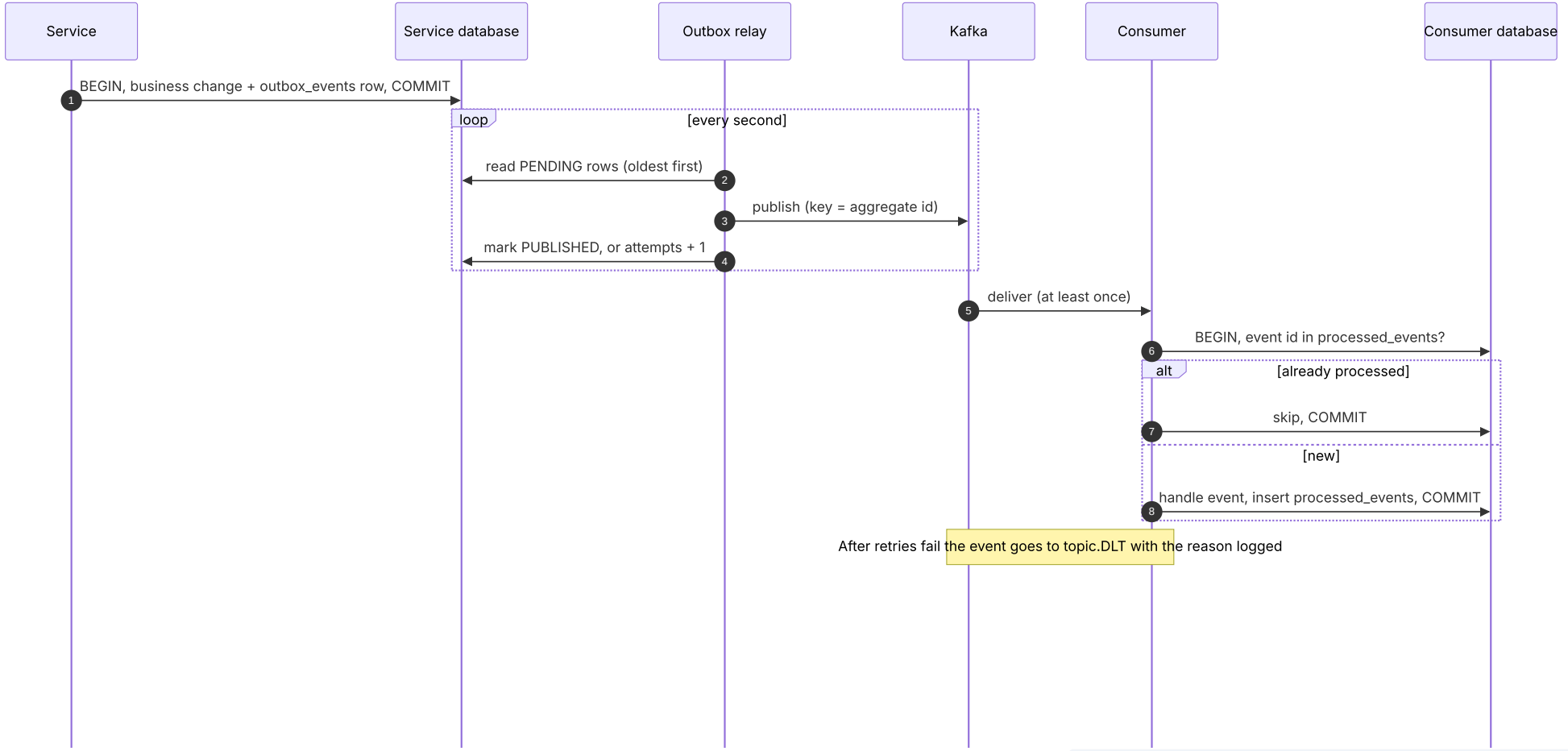


Source: `mermaid/38-review-and-trust.mmd`

5.10 Outbox and idempotent consumer

From SRS v1.1: 9.2, 9.4, ADR 0002

The mechanism under every dashed arrow above. Built in Phase 1 for Identity and Audit.



Source: `mermaid/39-outbox-and-idempotent-consumer.mmd`

6. Choices made and alternatives

Decision	Chosen	Alternatives	Why
Diagram format	Mermaid text files	PlantUML; draw.io	Lives in git, reviewed in PRs, drawn by GitHub. PlantUML needs Java to render; draw.io drifts from the code.
ERD detail	Main columns in diagrams, every column in tables	Every column in the diagrams	Full-column ERDs become wall-sized and unreadable on a page (kai's call).
Schema source	One Python file (`tools/schema.py`) generates ERDs and tables	Hand-written ERDs	The diagram and the column tables cannot disagree.
Cross-service links	Logical ids, drawn on the overview only	Foreign keys across databases	SRS 3.5 forbids cross-service foreign keys; each service must be deployable on its own.
Projections	Each service keeps the copies it needs, built from events	Call the owner over REST each time	No runtime dependency on another service being up; matches the event-driven design.
Ledger placement	Inside Payment	Separate Ledger service	A payment and its ledger entries must commit together (SRS 3.3).
Network layout	Public, private app, private data and ops subnets over 2 availability zones	One flat network; everything public behind a firewall	Only the load balancer is reachable from the internet; databases have no internet route (SRS 12, 15).
Cloud	AWS ap-south-1 (Mumbai), drawn with generic names	Azure Central India; GCP asia-south1	Storage adapter already targets S3 ap-south-1, data stays in India, managed Kafka (MSK) exists.
Web app	One Angular app with /shop, /seller, /admin	Three separate apps	One build and login, shared UI kit; lazy loading keeps each area small.
Rates	Commission on the category, fees and TCS/TDS in `rate_configs`, snapshots on order items	Hard-coded rates	SRS 8.2: rates are configuration with an effective-from date.

7. Open items found while drawing

Drawing everything in one place surfaced a few small gaps. None blocks Phase 2; each is a short task.

- **SUPPORT role row.** Identity's V1 migration seeds a `SUPPORT` role, which SRS v1.1 removed. A later migration should delete it and its description of ADMIN should change to "insights and exceptions".
- **fulfillment-events topic.** `libs/common` `Topics.java` has no `FULFILLMENT_EVENTS` constant yet. Add it when Fulfillment is built.
- **Tables added beyond SRS 8.4.** These diagrams add `wishlist_items`, `rate_configs`, `invoice_items`, Fulfillment's `order_item_projection`, Notification's `user_contacts`, Recommendation's `product_features` and Insights' `seller_action_items`. Each is marked in the column tables with the reason. SRS 8.4 could list them in v1.2.
- **Naming.** SRS 8.4 says `recovery_codes`; the built table is `mfa_recovery_codes`.
- **Planned schemas are a design, not a contract.** Column names for Phases 2 to 4 may change when each service is built; the Flyway migration is the final word, and these diagrams get regenerated.

8. Glossary

ADR	Architecture decision record: a short note in `docs/adr/` explaining one decision.
AWB	Air waybill: the courier's tracking number for a parcel.
CIDR	A way to write a range of network addresses, such as 10.0.16.0/20 (4,096 addresses).
Correlation id	One id that follows a user action through every service and event, for tracing logs.
DFD	Data-flow diagram: follows information between people, processes and stores.
DLT	Dead-letter topic: where Kafka events go after failing all retries.
ERD	Entity-relationship diagram: tables and the links between them.
Event	A record of a fact that already happened, such as OrderCreated, published on Kafka.
GMV	Gross merchandise value: total value of goods sold.
HSN	Harmonised System of Nomenclature code, which decides an item's GST rate.
Idempotent	Safe to repeat: doing it twice has the same effect as once.
JWKS	JSON Web Key Set: the public keys anyone can use to check our access tokens.
Logical id	An id copied from another service's data, with no foreign key behind it.
Mermaid	A text language for diagrams, drawn by GitHub and many editors.
NAT gateway	Lets private machines call out to the internet while blocking calls coming in.
NDR / RTO	Non-delivery report (a failed delivery attempt) / return to origin (parcel sent back to the seller).
Outbox	A table where a service saves events in the same transaction as its change, for a relay to publish.
Projection	A service's read-only copy of another service's data, built from events.
Saga	A multi-service business process made of local steps, with compensating steps on failure.

Soft delete	Marking a row deleted with `deleted_at` instead of removing it.
Subnet	A slice of the private network; firewall rules are written between subnets.
TCS / TDS	Tax collected at source (GST) / tax deducted at source (section 194-O), withheld from seller payouts.
TOTP	Time-based one-time password: the 6-digit code from an authenticator app.
VPC	Virtual private cloud: the company's own isolated network inside the cloud.
WAF	Web application firewall: blocks common attacks before they reach the app.

9. Appendix: Mermaid source

Every diagram above is plain text. Paste any block into `mermaid.live` or a GitHub Markdown file inside a ```` ```mermaid ```` fence to see and edit it. The ERD files are generated from `tools/schema.py`, so edit the schema there, not the .mmd.

mermaid/01-system-architecture.mmd

```
flowchart TB
  WEB["Angular web app<br/>/shop · /seller · /admin"]
  LB["Load balancer (HTTPS)"]
  GW["API gateway (Spring Cloud Gateway)<br/>JWT check · rate limit · CORS · correlation
id"]
  RD[("Redis<br/>rate limits · cache")]

  ID["Identity<br/>users · MFA<br/>sellers · KYC"]
  CAT["Catalog<br/>products<br/>listing checks"]
  COM["Commerce<br/>stock · cart<br/>orders"]
  PAY["Payment and<br/>settlement<br/>ledger · invoices"]
  FUL["Fulfillment<br/>shipments<br/>returns"]
  TR["Trust<br/>reviews<br/>scores"]
  SR["Search"]
  REC["Recommen-<br/>dation"]
  NOT["Notification"]
  INS["Insights<br/>dashboards<br/>reports"]
  AUD["Audit<br/>append-only log<br/>tickets"]

  K[("Kafka · domain events<br/>each service publishes through its outbox<br/>and
consumes the topics it needs")]
  PG[("PostgreSQL<br/>one database per service")]
  OS[("OpenSearch")]
  OB[("Object storage")]
  AIS["AI service<br/>Python FastAPI"]

  KYC(["KYC sandbox"])
  PSP(["Payment and<br/>payout provider"])
  CUR(["Courier"])
  MSG(["Email and SMS"])
  LLM(["LLM provider"])

  WEB --> LB --> GW
  GW -.- RD
  GW --> ID & CAT & COM & PAY & FUL & TR & SR & REC & NOT & INS & AUD
  ID & CAT & COM & PAY & FUL & TR & SR & REC & NOT & INS & AUD <--> K
  K ~~~ PG & OS & OB
  ID --> KYC
  PAY --> PSP
  FUL --> CUR
  NOT --> MSG
  REC --> AIS --> LLM
  SR --> OS
  PSP & CUR -. webhooks .-> GW
```

mermaid/02-deployment.mmd

```
flowchart LR
  U["Browsers"] --> LB["Load balancer<br/>HTTPS termination"]
  LB --> G1["Gateway replica 1"] & G2["Gateway replica 2"]
  G1 & G2 --> SV["Service replicas<br/>stateless, 2+ each"]
  SV --> PG[("PostgreSQL<br/>one database and login per service")]
  SV --> KF[("Kafka<br/>topics + .DLT dead-letter topics")]
  SV --> RD[("Redis")]
  SV --> OS[("OpenSearch")]
  SV --> OB[("Object storage")]
  G1 & G2 --> RD
  subgraph Observability
    PR["Prometheus metrics"]
    LG["Central logs<br/>with correlation id"]
    TRC["Traces"]
  end
  SV -.-> PR & LG & TRC
```

mermaid/03-component-identity.mmd

```
flowchart LR
  GW["API gateway"] --> SEC["Security filter chain<br/>JWT · roles · MFA claim"]

  subgraph Web["Controllers (HTTP)"]
    AC["AuthController<br/>/api/v1/auth"]
    UC["UserController<br/>/users/me"]
    MC["MfaController<br/>/users/me/mfa"]
    SC["SellerController<br/>/api/v1/sellers"]
    ADC["AdminController<br/>/api/v1/admin"]
    JW["JwksController<br/>/.well-known/jwks.json"]
  end

  subgraph Domain["Services (business rules)"]
    AS["AuthService<br/>login · refresh rotation · logout"]
    ATI["AccessTokenIssuer + SigningKeys<br/>RS256, rotatable"]
    MS["MfaService<br/>TOTP · recovery codes · challenges"]
    US["UserService"]
    SS["SellerService<br/>applications · admin decisions"]
    SVS["SellerVerificationService<br/>four KYC checks"]
    SDS["SellerDocumentService<br/>upload links · file checks"]
  end

  subgraph Jobs["Scheduled jobs"]
    SVW["SellerVerificationWorker<br/>every 2 s"]
    OR["OutboxRelay<br/>every 1 s"]
  end

  subgraph Out["Repositories and adapters"]
    REPO["JPA repositories<br/>+ OutboxWriter"]
    KP["KycProvider<br/>Mock · Sandbox"]
    ST["ObjectStorage<br/>Local · S3"]
  end

  DB[("identity_db")]
  K[("Kafka<br/>identity-events")]
  EXT(["sandbox.co.in"])
  OB[("Object storage")]

  SEC --> Web
  AC --> AS & MS
  UC --> US
  MC --> MS
  SC --> SS & SDS
  ADC --> SS & US & MS
  JW --> ATI
  AS --> ATI
  SS --> SVS
  SVW --> SVS
  AS & MS & US & SS & SVS & SDS --> REPO
  SVS --> KP --> EXT
  SDS --> ST --> OB
  REPO --> DB
  OR --> DB
  OR --> K
```

mermaid/04-component-web-app.mmd

```
flowchart TB
  subgraph App["Angular web app (one build)"]
    direction TB
    subgraph Core["core (loaded once)"]
      AUTH["AuthService<br/>access token in memory · refresh cookie"]
      INT["HTTP interceptors<br/>bearer token · correlation id · error mapping"]
      GUARD["Route guards<br/>role check · MFA check"]
      API["Typed API clients<br/>one per backend service"]
    end
    subgraph Shop["/shop (lazy loaded)"]
      S1["Home and recommendations"]
      S2["Search and product page"]
      S3["Compare (up to 4) and AI assistant"]
      S4["Cart, checkout, payment"]
      S5["My orders, returns, reviews, tickets"]
    end
    subgraph Seller["/seller (lazy, role SELLER)"]
      L1["Apply and verification status"]
      L2["Products, images, prices"]
      L3["Inventory and low-stock"]
      L4["Seller orders and dispatch"]
      L5["Dashboard and settlements"]
    end
    subgraph Admin["/admin (lazy, role ADMIN + MFA)"]
      A1["Insights dashboard"]
      A2["Exceptions queue<br/>sellers · listings · reviews · disputes"]
      A3["Audit search and system health"]
      A4["Reports, categories, brands, tickets"]
    end
    SHARED["shared UI kit<br/>tables · forms · charts"]
  end
  Shop & Seller & Admin --> API
  Shop & Seller & Admin --> SHARED
  GUARD --> Seller & Admin
  API --> INT --> GW["API gateway"]
```

mermaid/05-dfd-context.mmd

```
flowchart LR
  C(["Customer"])
  S(["Seller"])
  A(["Admin"])
  BB(("0<br/>BigBazaar<br/>platform"))
  KYC(["KYC provider"])
  PSP(["Payment and<br/>payout provider"])
  CUR(["Courier"])
  MSG(["Email and SMS"])
  LLM(["LLM provider"])

  C <-- "IN: sign-up, searches, cart, orders, payments, returns, reviews,
prompts<br/>OUT: products, recommendations, order status, invoices, refunds" --> BB
  S <-- "IN: application, KYC details, products, stock, packing<br/>OUT: verification
result, seller orders, labels, dashboard, settlements" --> BB
  A <-- "IN: exception decisions with reason, categories, brands, report
requests<br/>OUT: insights, alerts, audit log, reports" --> BB
  BB <-- "OUT: GSTIN, PAN, bank, DigiLocker checks<br/>IN: registry results" --> KYC
  BB <-- "OUT: payment orders, refunds, payouts<br/>IN: payment webhooks, payout status"
--> PSP
  BB <-- "OUT: shipment and pickup bookings<br/>IN: AWB, labels, tracking webhooks" -->
CUR
  BB -- "OUT: email, SMS codes" --> MSG
  BB <-- "OUT: prompts with real catalog candidates<br/>IN: filters, explanations,
summaries" --> LLM
```

mermaid/06-dfd-level1.mmd

```
flowchart TB
  P1["1 Manage identity and sellers<br/><small>customer: account · seller: application,
KYC<br/>admin: verification exceptions</small>"]
  P2["2 Manage catalog<br/><small>seller: listings, prices<br/>admin: doubtful
listings</small>"]
  P3["3 Take orders, hold stock<br/><small>customer: cart, checkout<br/>seller: stock,
packing</small>"]
  P4["4 Take payments, settle<br/><small>customer: payment<br/>provider:
webhooks</small>"]
  P5["5 Ship and return<br/><small>customer: returns<br/>courier: tracking</small>"]
  P6["6 Reviews and trust<br/><small>customer: reviews<br/>admin: flagged
reviews</small>"]
  P7["7 Search and recommend<br/><small>customer: queries, prompts</small>"]

  D1[("D1 identity_db")]
  D2[("D2 catalog_db")]
  D3[("D3 commerce_db")]
  D4[("D4 payment_db")]
  D5[("D5 fulfillment_db")]
  D6[("D6 trust_db")]
  D7[("D7 OpenSearch,<br/>recommendation_db")]

  EV{{"Kafka event stream<br/><small>processes 2 to 7 also read it back to build their
projections</small>"}}

  P8["8 Build insights and audit"]
  P9["9 Notify"]
  D8[("D8 insights_db, platform_db")]
  D9[("D9 notification_db")]
  A(["Admin and seller dashboards"])
  C(["Customer and seller inboxes"])

  P1 <--> D1
  P2 <--> D2
  P3 <--> D3
  P4 <--> D4
  P5 <--> D5
  P6 <--> D6
  P7 <--> D7

  D1 & D2 & D3 & D4 & D5 & D6 -- "facts via outbox" --> EV
  EV -- "all events" --> P8
  EV --> P9
  P8 <--> D8
  P9 <--> D9
  P8 -- "totals, alerts, reports" --> A
  P9 -- "email, SMS, in-app" --> C
```

mermaid/07-event-flow.mmd

```
flowchart LR
  ID["Identity"] --> T1[["identity-events<br/>→ Catalog (seller projection),
Notification"]]
  CAT["Catalog"] --> T2[["product-events<br/>→ Commerce (product projection), Search,
Recommendation"]]
  COM["Commerce"] --> T3[["commerce-events<br/>→ Payment, Fulfillment, Trust, Search,
Recommendation, Notification"]]
  PAY["Payment"] --> T4[["payment-events<br/>→ Commerce (confirm or release),
Fulfillment, Notification"]]
  FUL["Fulfillment"] --> T5[["fulfillment-events<br/>→ Commerce, Payment (holds,
refunds), Trust, Notification"]]
  TR["Trust"] --> T6[["trust-events<br/>→ Catalog, Search, Recommendation"]]
  REC["Recommendation"] --> T7[["recommendation-events<br/>→ Recommendation (activity)"]]
  NOT["Notification"] --> T8[["notification-events<br/>→ delivery history"]]
  T1 & T2 & T3 & T4 & T5 & T6 & T7 & T8 --> ALL["Insights and Audit<br/>consume every
topic"]
```

mermaid/08-network.mmd

```
%% layout: elk (drawn with the ELK engine in the PDF build)
flowchart LR
  subgraph NET["Internet"]
    USR(["Browsers"])
    HOOK(["Payment and courier<br/>webhooks"])
  end

  subgraph EDGE["Edge (managed)"]
    DNS["DNS<br/>bigbazaar.in"]
    CDN["CDN + WAF<br/>Angular static files"]
  end

  subgraph PUB["Public subnet 10.0.0.0/20"]
    LB["Load balancer + WAF<br/>:443 HTTPS<br/>TLS ends here"]
  end

  subgraph APP["Private app subnet 10.0.16.0/20 · no public IPs"]
    GW["API gateway ×2<br/>:8080"]
    SVC["Java services, 2+ replicas each<br/>identity :8081 · audit :8091<br/>catalog
:8082 · commerce :8083<br/>payment :8084 · fulfillment :8085<br/>notification :8086 ·
trust :8087<br/>search :8088 · recommendation :8090<br/>insights :8092"]
    AI["AI service ×2<br/>:8000"]
  end

  subgraph DATA["Private data subnet 10.0.32.0/20"]
    PG[("PostgreSQL<br/>primary + standby<br/>:5432")]
    KF[("Kafka ×3 brokers<br/>:9093 TLS")]
    RD[("Redis primary + replica<br/>:6379 TLS")]
    OS[("OpenSearch ×3<br/>:9200 HTTPS")]
  end

  subgraph OUT["Leaving the network"]
    S3E["Private endpoint"]
    OBJ[("Object storage<br/>docs · images · invoices")]
    NAT["NAT gateway<br/>outbound only"]
    EXT(["KYC, payment, payout,<br/>courier, SMS, email, LLM APIs"])
  end

  subgraph OPS["Ops subnet 10.0.48.0/24"]
    VPN["VPN endpoint<br/>admins only"]
    MON["Prometheus · Grafana<br/>logs and traces"]
  end

  USR -- "443" --> DNS
  USR -- "443" --> CDN
  USR -- "443 /api" --> LB
  HOOK -- "443, signed" --> LB
  LB -- "8080" --> GW
  GW -- "808x, 809x" --> SVC
  SVC -- "8000" --> AI
  GW -- "6379" --> RD
```

```
  SVC -- "5432, own login" --> PG
  SVC -- "9093" --> KF
  SVC -- "6379" --> RD
  SVC -- "9200" --> OS
  SVC -- "443" --> S3E --> OBJ
  SVC -- "443" --> NAT
  AI -- "443" --> NAT
  NAT -- "443" --> EXT
  VPN -.-> MON
  MON -. "scrape /actuator" .-> SVC
```

mermaid/20-erd-00-overview.mmd

```
flowchart LR
  identity_db[("identity_db<br/><small>Identity</small>")]
  catalog_db[("catalog_db<br/><small>Catalog</small>")]
  commerce_db[("commerce_db<br/><small>Commerce</small>")]
  payment_db[("payment_db<br/><small>Payment and settlement</small>")]
  fulfillment_db[("fulfillment_db<br/><small>Fulfillment</small>")]
  notification_db[("notification_db<br/><small>Notification</small>")]
  trust_db[("trust_db<br/><small>Trust</small>")]
  recommendation_db[("recommendation_db<br/><small>Recommendation</small>")]
  insights_db[("insights_db<br/><small>Insights</small>")]
  platform_db[("platform_db<br/><small>Audit (and support tickets)</small>")]
  catalog_db -. "sellers" .-> identity_db
  commerce_db -. "categories, product_variants, products" .-> catalog_db
  commerce_db -. "sellers, users" .-> identity_db
  fulfillment_db -. "product_variants" .-> catalog_db
  fulfillment_db -. "order_items, orders, seller_orders" .-> commerce_db
  fulfillment_db -. "sellers, users" .-> identity_db
  insights_db -. "products" .-> catalog_db
  insights_db -. "sellers, users" .-> identity_db
  notification_db -. "users" .-> identity_db
  payment_db -. "order_items, orders, seller_orders" .-> commerce_db
  payment_db -. "sellers, users" .-> identity_db
  platform_db -. "orders" .-> commerce_db
  platform_db -. "users" .-> identity_db
  recommendation_db -. "brands, categories, products" .-> catalog_db
  recommendation_db -. "users" .-> identity_db
  trust_db -. "product_variants, products" .-> catalog_db
  trust_db -. "order_items" .-> commerce_db
  trust_db -. "sellers, users" .-> identity_db
  classDef built fill:#d8f3dc,stroke:#2d6a4f
  class identity_db,platform_db built
```

mermaid/20-erd-01-identity-db.mmd

```
erDiagram
  users {
    uuid id PK
    varchar name
    varchar email UK
    varchar phone
    varchar status
  }
  roles {
    varchar name PK
  }
  user_roles {
    uuid user_id PK, FK
    varchar role PK, FK
  }
  addresses {
    uuid id PK
    uuid user_id FK
    varchar city
    varchar state
    varchar postal_code
  }
  phone_verifications {
    uuid id PK
    uuid user_id FK
    varchar phone
    timestamptz expires_at
    timestamptz verified_at
  }
  user_mfa {
    uuid user_id PK, FK
    varchar status
  }
  mfa_recovery_codes {
    uuid id PK
    uuid user_id FK
    timestamptz used_at
  }
  mfa_challenges {
    uuid id PK
    uuid user_id FK
    timestamptz expires_at
  }
  refresh_tokens {
    uuid id PK
    uuid user_id FK
    uuid family_id
    timestamptz expires_at
    timestamptz revoked_at
  }
  sellers {
```

```
    uuid id PK
    uuid user_id FK, UK
    varchar business_name
    varchar tax_id
    varchar status
  }
  seller_documents {
    uuid id PK
    uuid seller_id FK
    varchar document_type
    varchar storage_key UK
    varchar status
  }
  seller_verifications {
    uuid id PK
    uuid seller_id FK
    varchar status
    int attempts
  }
  seller_verification_checks {
    bigserial id PK
    uuid verification_id FK
    varchar check_type
    varchar outcome
  }
  seller_status_history {
    bigserial id PK
    uuid seller_id FK
    varchar to_status
    uuid actor_id
    timestamptz changed_at
  }
  users ||--o{ user_roles : "user_id"
  roles ||--o{ user_roles : "role"
  users ||--o{ addresses : "user_id"
  users ||--o{ phone_verifications : "user_id"
  users ||--o| user_mfa : "user_id"
  users ||--o{ mfa_recovery_codes : "user_id"
  users ||--o{ mfa_challenges : "user_id"
  users ||--o{ refresh_tokens : "user_id"
  users ||--o| sellers : "user_id"
  sellers ||--o{ seller_documents : "seller_id"
  sellers ||--o{ seller_verifications : "seller_id"
  seller_verifications ||--o{ seller_verification_checks : "verification_id"
  sellers ||--o{ seller_status_history : "seller_id"
```

mermaid/20-erd-02-catalog-db.mmd

```
erDiagram
  verified_seller_projection {
    uuid seller_id PK "logical id"
    varchar business_name
    varchar gstin
    varchar state_code
    varchar status
  }
  categories {
    uuid id PK
    uuid parent_id FK
    varchar name
    boolean returnable
    int return_window_days
    numeric commission_rate
  }
  brands {
    uuid id PK
    varchar name UK
    varchar status
  }
  products {
    uuid id PK
    uuid seller_id "logical id"
    uuid category_id FK
    uuid brand_id FK
    varchar title
    varchar status
    varchar hsn_code
    numeric gst_rate
  }
  product_variants {
    uuid id PK
    uuid product_id FK
    varchar sku UK
    jsonb attributes
    numeric price
    numeric mrp
  }
  product_images {
    uuid id PK
    uuid product_id FK
    varchar storage_key
  }
  price_history {
    bigserial id PK
    uuid variant_id FK
    numeric new_price
    timestamptz changed_at
  }
  listing_checks {
```

```
    bigserial id PK
    uuid product_id FK
    varchar rule
    varchar outcome
  }
  categories |o--o{ categories : "parent_id"
  categories ||--o{ products : "category_id"
  brands |o--o{ products : "brand_id"
  products ||--o{ product_variants : "product_id"
  products ||--o{ product_images : "product_id"
  product_variants |o--o{ product_images : "variant_id"
  product_variants ||--o{ price_history : "variant_id"
  products ||--o{ listing_checks : "product_id"
```

mermaid/20-erd-03-commerce-db.mmd

```
erDiagram
  product_projection {
    uuid variant_id PK "logical id"
    uuid product_id "logical id"
    uuid seller_id "logical id"
    numeric price
    varchar status
  }
  inventory {
    uuid variant_id PK, FK
    uuid seller_id "logical id"
    int on_hand_quantity
    int reserved_quantity
  }
  stock_reservations {
    uuid id PK
    uuid variant_id FK
    uuid order_id FK
    int quantity
    varchar status
    timestamptz expires_at
  }
  stock_movements {
    bigserial id PK
    uuid variant_id FK
    varchar movement_type
    int quantity_delta
    timestamptz created_at
  }
  carts {
    uuid id PK
    uuid user_id UK "logical id"
  }
  cart_items {
    uuid id PK
    uuid cart_id FK
    uuid variant_id FK
    int quantity
  }
  wishlists {
    uuid id PK
    uuid user_id UK "logical id"
  }
  wishlist_items {
    uuid id PK
    uuid wishlist_id FK
    uuid product_id "logical id"
    numeric price_at_add
  }
  orders {
    uuid id PK
```

```
    varchar order_number UK
    uuid user_id "logical id"
    varchar status
    numeric total_amount
  }
  seller_orders {
    uuid id PK
    uuid order_id FK
    uuid seller_id "logical id"
    varchar status
    numeric total_amount
    timestamptz dispatch_deadline_at
  }
  order_items {
    uuid id PK
    uuid order_id FK
    uuid seller_order_id FK
    uuid variant_id "logical id"
    int quantity
    numeric unit_price
    varchar status
  }
  order_status_history {
    bigserial id PK
    uuid order_id FK
    varchar to_status
    timestamptz changed_at
  }
  idempotency_keys {
    uuid user_id PK "logical id"
    varchar idempotency_key PK
    uuid order_id FK
  }
  product_projection ||--o| inventory : "variant_id"
  inventory ||--o{ stock_reservations : "variant_id"
  orders ||--o{ stock_reservations : "order_id"
  inventory ||--o{ stock_movements : "variant_id"
  carts ||--o{ cart_items : "cart_id"
  product_projection ||--o{ cart_items : "variant_id"
  wishlists ||--o{ wishlist_items : "wishlist_id"
  orders ||--o{ seller_orders : "order_id"
  orders ||--o{ order_items : "order_id"
  seller_orders ||--o{ order_items : "seller_order_id"
  orders ||--o{ order_status_history : "order_id"
  seller_orders |o--o{ order_status_history : "seller_order_id"
  orders |o--o{ idempotency_keys : "order_id"
```

mermaid/20-erd-04-payment-db.mmd

```
erDiagram
  payments {
    uuid id PK
    uuid order_id UK "logical id"
    uuid user_id "logical id"
    numeric amount
    varchar status
  }
  payment_attempts {
    uuid id PK
    uuid payment_id FK
    varchar method
    varchar status
  }
  payment_webhook_events {
    uuid id PK
    varchar provider UK
    varchar provider_event_id UK
    timestamptz processed_at
  }
  refunds {
    uuid id PK
    uuid payment_id FK
    uuid seller_order_id "logical id"
    numeric amount
    varchar reason
    varchar status
  }
  ledger_accounts {
    uuid id PK
    varchar code UK
    varchar owner_type
    uuid owner_id UK "logical id"
  }
  ledger_entries {
    bigserial id PK
    uuid transaction_id
    uuid account_id FK
    numeric debit_amount
    numeric credit_amount
    varchar reference_type
  }
  rate_configs {
    uuid id PK
    varchar rate_type
    date effective_from
  }
  settlements {
    uuid id PK
    uuid seller_id UK "logical id"
    date cycle_start UK
```

```
    date cycle_end
    numeric net_payable
    varchar status
  }
  settlement_items {
    uuid id PK
    uuid settlement_id FK
    uuid seller_order_id UK "logical id"
    numeric net_amount
  }
  payouts {
    uuid id PK
    uuid settlement_id FK, UK
    uuid seller_id "logical id"
    numeric amount
    varchar status
  }
  invoices {
    uuid id PK
    varchar invoice_number UK
    uuid seller_order_id UK "logical id"
    uuid seller_id "logical id"
    varchar tax_type
    numeric total_amount
  }
  invoice_items {
    uuid id PK
    uuid invoice_id FK
    uuid order_item_id "logical id"
    varchar hsn_code
    numeric gst_rate
  }
  payments ||--o{ payment_attempts : "payment_id"
  payments ||--o{ refunds : "payment_id"
  ledger_accounts ||--o{ ledger_entries : "account_id"
  ledger_entries |o--o{ ledger_entries : "reverses_entry_id"
  settlements ||--o{ settlement_items : "settlement_id"
  settlements ||--o| payouts : "settlement_id"
  invoices ||--o{ invoice_items : "invoice_id"
```

mermaid/20-erd-05-fulfillment-db.mmd

```
erDiagram
  serviceable_pincodes {
    varchar pincode PK
    varchar courier PK
    int estimated_days
  }
  order_item_projection {
    uuid order_item_id PK "logical id"
    uuid seller_order_id "logical id"
    uuid user_id "logical id"
    int return_window_days
  }
  shipments {
    uuid id PK
    uuid seller_order_id UK "logical id"
    uuid order_id "logical id"
    uuid seller_id "logical id"
    varchar status
    varchar courier
    varchar awb_number UK
  }
  shipment_events {
    bigserial id PK
    uuid shipment_id FK
    varchar status
    timestamptz occurred_at
  }
  courier_webhook_events {
    uuid id PK
    varchar provider UK
    varchar provider_event_id UK
    timestamptz processed_at
  }
  return_requests {
    uuid id PK
    uuid order_item_id FK
    uuid seller_order_id "logical id"
    uuid user_id "logical id"
    varchar reason
    varchar status
  }
  reverse_pickups {
    uuid id PK
    uuid return_request_id FK, UK
    varchar awb_number UK
    varchar status
  }
  shipments ||--o{ shipment_events : "shipment_id"
  courier_webhook_events |o--o{ shipment_events : "courier_webhook_event_id"
```

```
  order_item_projection ||--o{ return_requests : "order_item_id"
  return_requests ||--o| reverse_pickups : "return_request_id"
```

mermaid/20-erd-06-notification-db.mmd

```
erDiagram
  user_contacts {
    uuid user_id PK "logical id"
    varchar email
    varchar phone
  }
  notifications {
    uuid id PK
    uuid user_id FK
    varchar channel
    varchar template
    varchar status
  }
  delivery_attempts {
    bigserial id PK
    uuid notification_id FK
    varchar outcome
  }
  preferences {
    uuid user_id PK, FK
    varchar channel PK
    varchar topic PK
  }
  user_contacts ||--o{ notifications : "user_id"
  notifications ||--o{ delivery_attempts : "notification_id"
  user_contacts ||--o{ preferences : "user_id"
```

mermaid/20-erd-07-trust-db.mmd

```
erDiagram
  delivered_item_projection {
    uuid order_item_id PK "logical id"
    uuid user_id "logical id"
    uuid product_id "logical id"
    uuid seller_id "logical id"
    timestamptz delivered_at
  }
  reviews {
    uuid id PK
    uuid order_item_id FK, UK
    uuid user_id "logical id"
    uuid product_id "logical id"
    smallint rating
    varchar status
  }
  review_flags {
    uuid id PK
    uuid review_id FK
    varchar rule
    varchar status
  }
  review_summaries {
    uuid product_id PK "logical id"
    timestamptz generated_at
  }
  product_scores {
    uuid product_id PK "logical id"
    numeric score
  }
  seller_scores {
    uuid seller_id PK "logical id"
    numeric score
    numeric cancellation_rate
    numeric late_dispatch_rate
    numeric return_rate
    boolean flagged
  }
  score_history {
    bigserial id PK
    varchar subject_type
    uuid subject_id
    numeric new_score
    timestamptz recorded_at
  }
  delivered_item_projection ||--o| reviews : "order_item_id"
  reviews ||--o{ review_flags : "review_id"
```

mermaid/20-erd-08-recommendation-db.mmd

```
erDiagram
  product_features {
    uuid product_id PK "logical id"
    uuid category_id "logical id"
    numeric min_price
    boolean active
  }
  user_activity {
    bigserial id PK
    uuid user_id "logical id"
    varchar activity_type
    uuid product_id FK
    timestamptz occurred_at
  }
  user_preferences {
    uuid user_id PK "logical id"
  }
  ai_requests {
    uuid id PK
    varchar request_type
    int latency_ms
    varchar status
  }
  recommendation_logs {
    uuid id PK
    uuid user_id "logical id"
    varchar context
    varchar algorithm_version
  }
  recommendation_items {
    bigserial id PK
    uuid log_id FK
    uuid product_id FK
    int rank
    jsonb reasons
  }
  product_similarity {
    uuid product_id PK, FK
    uuid similar_product_id PK, FK
    numeric score
  }
  product_features |o--o{ user_activity : "product_id"
  ai_requests |o--o{ recommendation_logs : "ai_request_id"
  recommendation_logs ||--o{ recommendation_items : "log_id"
  product_features ||--o{ recommendation_items : "product_id"
  product_features ||--o{ product_similarity : "product_id"
  product_features ||--o{ product_similarity : "similar_product_id"
```

mermaid/20-erd-09-insights-db.mmd

```
erDiagram
  daily_platform_totals {
    date day PK
    int orders_count
    numeric gmv
  }
  daily_seller_totals {
    date day PK
    uuid seller_id PK "logical id"
    int orders_count
    numeric revenue
  }
  daily_product_totals {
    date day PK
    uuid product_id PK "logical id"
    uuid seller_id "logical id"
    int units_sold
    numeric revenue
  }
  seller_action_items {
    uuid id PK
    uuid seller_id "logical id"
    varchar item_type
    uuid reference_id
    boolean open
  }
  alert_feed {
    uuid id PK
    varchar alert_type
    varchar subject_type
    uuid subject_id
    varchar status
  }
  report_runs {
    uuid id PK
    varchar report_type
    date period_from
    date period_to
    varchar format
    varchar status
  }
```

mermaid/20-erd-10-platform-db.mmd

```
erDiagram
  audit_events {
    bigserial id PK
    uuid event_id UK
    varchar event_type
    varchar source_service
    uuid actor_id
    varchar entity_type
    varchar entity_id
    uuid correlation_id
    timestamptz occurred_at
  }
  support_tickets {
    uuid id PK
    varchar ticket_number UK
    uuid user_id "logical id"
    varchar subject
    varchar status
  }
  ticket_messages {
    bigserial id PK
    uuid ticket_id FK
    uuid author_id "logical id"
    varchar author_role
    timestamptz created_at
  }
  support_tickets ||--o{ ticket_messages : "ticket_id"
```

mermaid/20-erd-11-search-index.mmd

```
erDiagram
  products_index {
    keyword product_id PK
    text title
    keyword category_path
    keyword brand
    keyword seller_id
    scaled_float min_price
    boolean in_stock
    keyword status
  }
```

mermaid/30-request-through-gateway.mmd

```
sequenceDiagram
  autonumber
  participant B as Browser (Angular)
  participant G as API gateway
  participant R as Redis
  participant S as Service
  participant DB as Service database
  B->>G: HTTPS request + Bearer access token
  G->>G: add X-Correlation-Id if missing
  G->>R: rate-limit counter for this user or IP
  alt over the limit
    G-->>B: 429 Too Many Requests (ApiError)
  end
  G->>G: verify RS256 signature with cached JWKS, check expiry
  alt token missing or invalid on a protected path
    G-->>B: 401 Unauthorized (ApiError)
  end
  G->>S: forward request + correlation id
  S->>S: check role, MFA claim and ownership
  S->>DB: read or write (one transaction, outbox row if a fact changed)
  DB-->>S: result
  S-->>G: JSON response
  G-->>B: response with X-Correlation-Id
```

mermaid/31-login-with-mfa.mmd

```
sequenceDiagram
  autonumber
  actor U as User
  participant W as Web app
  participant G as Gateway
  participant I as Identity
  participant DB as identity_db
  U->>W: email + password
  W->>G: POST /api/v1/auth/login
  G->>I: forward (public path, rate limited)
  I->>DB: load user, check status and lockout
  I->>I: BCrypt check
  alt wrong password
    I->>DB: failed_login_attempts + 1 (lock after the limit)
    I-->>W: 401 INVALID_CREDENTIALS
  else MFA is active (always for admins)
    I->>DB: insert mfa_challenges (hashed token, 5 min)
    I-->>W: 200 status MFA_REQUIRED + mfaToken
    U->>W: 6-digit code from authenticator app
    W->>G: POST /api/v1/auth/mfa/verify
    G->>I: forward
    I->>I: TOTP check, reject a reused time step
    I->>DB: consume challenge, insert refresh_tokens (mfa_verified = true)
    I-->>W: access token (RS256, 15 min, amr = [pwd, mfa]) + refresh token
  else no MFA
    I->>DB: insert refresh_tokens
    I-->>W: status AUTHENTICATED, access token (amr = [pwd]) + refresh token
  end
  Note over W,I: Later, POST /auth/refresh rotates the refresh token. Reusing an old one
revokes the whole family.
```

mermaid/32-checkout-and-payment.mmd

```
sequenceDiagram
  autonumber
  actor Cu as Customer
  participant F as Fulfillment
  participant Co as Commerce
  participant K as Kafka
  participant P as Payment
  participant PSP as Payment provider
  Cu->>F: GET serviceability for pincode
  F-->>Cu: deliverable, estimated date
  Cu->>Co: POST /checkout (Idempotency-Key)
  Note over Co: key seen before? return the saved order<br/>revalidate price and
stock<br/>ONE transaction: order, seller_orders,<br/>items with snapshots, reservations
(15 min),<br/>outbox row OrderCreated
  Co-->>Cu: 201 order PAYMENT_PENDING
  Co--)K: OrderCreated
  K--)P: OrderCreated
  P->>P: create payment PENDING, provider order
  Cu->>P: pay
  P->>PSP: create payment
  PSP-->>Cu: UPI / card page
  PSP->>P: signed webhook (deduplicated)
  P->>PSP: verify status on the backend
  alt success
    Note over P: payment CAPTURED<br/>ledger entries, outbox row
    P--)K: PaymentSucceeded
    K--)Co: PaymentSucceeded
    Note over Co: order CONFIRMED, reservation CONFIRMED,<br/>stock movement, dispatch
deadlines set
  else failure or timeout
    P--)K: PaymentFailed
    K--)Co: PaymentFailed
    Note over Co: order PAYMENT_FAILED,<br/>release reservation, stock movement
  end
```

mermaid/33-seller-verification.mmd

```
sequenceDiagram
  autonumber
  actor S as Seller
  participant I as Identity
  participant W as Verification worker
  participant KYC as KYC provider
  participant K as Kafka
  actor A as Admin
  S->>I: POST /sellers/applications (GSTIN, PAN, date, bank)
  I->>I: encrypt KYC inputs, seller PENDING_VERIFICATION
  I--)K: SellerApplicationSubmitted
  S->>I: connect DigiLocker
  I->>I: verification READY, SellerVerificationStarted
  loop every 2 s, picks due verifications
    W->>KYC: GSTIN, PAN, penny drop, DigiLocker identity
    KYC-->>W: results with registry names
    W->>I: store seller_verification_checks
  end
  alt every check passes and names match
    I->>I: seller APPROVED by SYSTEM, grant SELLER role
    I--)K: SellerApproved
  else a check fails (attempt 1 to 3)
    I-->>S: which check failed and why, correct and retry
  else still failing or provider down after retries
    I->>I: seller UNDER_REVIEW
    A->>I: POST /admin/sellers/{id}/decisions (MFA, reason)
    I--)K: SellerApproved or SellerRejected
  end
  Note over I,K: Catalog builds its verified_seller_projection from these events. Audit
records each step.
```

mermaid/34-product-publication.mmd

```
sequenceDiagram
  autonumber
  actor S as Seller
  participant C as Catalog
  participant AI as AI service
  participant K as Kafka
  participant Co as Commerce
  participant SR as Search
  actor A as Admin
  S->>C: POST /products (HSN, GST rate, legal fields, variants)
  C->>C: seller in verified_seller_projection?
  C->>C: status PENDING_CHECKS
  C->>C: rules: banned items and words, required details, image size, brand
  C->>AI: category fit and wording help
  AI-->>C: suggestion with confidence
  alt all pass
    C->>C: ACTIVE, ProductPublished
  else clear failure
    C->>C: REJECTED with fixable reason, ListingRejected
  else unsure
    C->>C: PENDING_APPROVAL
    A->>C: approve or reject with reason (MFA)
  end
  C--)K: ProductPublished
  K--)Co: update product_projection
  K--)SR: index product
```

mermaid/35-shipping-and-delivery.mmd

```
sequenceDiagram
  autonumber
  actor S as Seller
  participant Co as Commerce
  participant K as Kafka
  participant F as Fulfillment
  participant CR as Courier
  participant P as Payment
  participant T as Trust
  S->>Co: mark seller order PACKED (before dispatch deadline)
  Co--)K: OrderPacked
  K--)F: OrderPacked
  F->>CR: book shipment
  CR-->>F: AWB + label
  F--)K: ShipmentBooked
  CR->>F: webhook picked up (signed, deduplicated)
  F--)K: OrderShipped
  CR->>F: webhooks in transit, out for delivery
  alt delivered
    CR->>F: webhook delivered
    F--)K: OrderDelivered
    K--)Co: seller order DELIVERED, return window starts
    K--)P: start return-window hold
    K--)T: review eligibility opens
  else delivery failed
    F--)K: DeliveryFailed, customer picks a new attempt
    opt too many failures
      F--)K: ReturnedToOrigin
      K--)Co: cancel seller order
      K--)P: refund customer
    end
  end
  Note over Co: Deadline missed without PACKED: Commerce cancels the seller order,
OrderCancelled, Payment refunds.
```

mermaid/36-return-and-refund.mmd

```
sequenceDiagram
  autonumber
  actor Cu as Customer
  participant F as Fulfillment
  participant CR as Courier
  participant K as Kafka
  participant Co as Commerce
  participant P as Payment
  participant PSP as Payment provider
  Cu->>F: POST /returns (order item, reason, photos)
  F->>F: delivered? inside return window? category returnable?
  F--)K: ReturnRequested
  F->>CR: book reverse pickup
  CR->>F: webhook picked up
  F--)K: ReturnPickedUp
  CR->>F: webhook delivered to seller
  F->>F: seller marks resellable or not
  F--)K: ReturnReceived
  K--)Co: restock if resellable, stock movement RESTOCK
  K--)P: ReturnReceived
  P->>PSP: refund to original method
  PSP-->>P: refund done
  P->>P: refund COMPLETED, reversing ledger entries
  P--)K: RefundCompleted
```

mermaid/37-settlement-and-payout.mmd

```
sequenceDiagram
  autonumber
  participant K as Kafka
  participant P as Payment
  participant L as Ledger (in Payment)
  participant J as Settlement job
  participant PO as Payout provider
  K--)P: PaymentSucceeded
  P->>L: debit customer payments, credit seller payable, commission, fees
  P->>P: GST invoice per seller order (CGST+SGST or IGST), PDF to storage
  P--)K: InvoiceGenerated
  K--)P: OrderDelivered
  P->>P: hold until the return window closes
  Note over P: window closed and no open return: amount becomes payable
  J->>J: run on the settlement cycle (locked, idempotent)
  J->>L: per seller: payable - commission - fees - TCS - TDS - refunds
  J->>P: settlement + settlement_items
  J->>PO: payout
  PO-->>J: completed
  J->>L: debit seller payable, credit payouts
  J--)K: PayoutCompleted
```

mermaid/38-review-and-trust.mmd

```
sequenceDiagram
  autonumber
  actor Cu as Customer
  participant K as Kafka
  participant T as Trust
  actor A as Admin
  K--)T: OrderDelivered, store delivered_item_projection
  Cu->>T: POST /reviews (order item, rating, text)
  T->>T: delivered to this user? one review per item?
  T->>T: fake-review rules: burst, seller-linked, near-duplicate text
  alt clean
    T--)K: ReviewPublished
  else suspicious
    T--)K: ReviewFlagged
    A->>T: uphold or dismiss with reason (MFA)
  end
  T->>T: recalculate product and seller score, score_history row
  T--)K: ProductTrustScoreUpdated, SellerTrustScoreUpdated
  Note over K: Catalog, Search, Recommendation and Insights consume the score updates
```

mermaid/39-outbox-and-idempotent-consumer.mmd

```
sequenceDiagram
  autonumber
  participant S as Service
  participant DB as Service database
  participant R as Outbox relay
  participant K as Kafka
  participant C as Consumer
  participant CDB as Consumer database
  S->>DB: BEGIN, business change + outbox_events row, COMMIT
  loop every second
    R->>DB: read PENDING rows (oldest first)
    R->>K: publish (key = aggregate id)
    R->>DB: mark PUBLISHED, or attempts + 1
  end
  K->>C: deliver (at least once)
  C->>CDB: BEGIN, event id in processed_events?
  alt already processed
    C->>CDB: skip, COMMIT
  else new
    C->>CDB: handle event, insert processed_events, COMMIT
  end
  Note over C,K: After retries fail the event goes to topic.DLT with the reason logged
```